МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра информационных технологий в атомной энергетике (ИТвАЭ)

КУРСОВАЯ РАБОТА

по дисциплине «Автоматизированные системы управления элементами промышленных и административных объектов»

Тема: « Система по поддержанию температуры в оранжерее »

Студент Астахов Сергей Павлович

Группа ИКБО-50-23

(подпись студента)

Работа представлена к защите 23.05.26 (дата)

Руководитель Щербаков К.В

Работа допущена к защите 23.05.26 (дата)

(подпись и ФИО руководителя)

Оценка по итогам защиты отлично

/ Щербаков Кирилл Владимирович, старший преподаватель

/

(подписи, дата, ф.и.о., должность, звание, уч. степень двух преподавателей, принявших защиту)

РТУ МИРЭА 2026 г.

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра информационных технологий в атомной энергетике (ИТвАЭ)

Утверждаю
Заведующий кафедрой ИТвАЭ
_____ Боридько С.И.
(подпись)
28 августа 2025 г.

ЗАДАНИЕ

на выполнение курсовой работы

по дисциплине «Автоматизированные системы управления элементами промышленных и административных объектов»

Студент Астахов Сергей Павлович Группа ИКБО-50-23

Тема «Система по поддержанию температуры в оранжереи».

Цель курсовой работы:
Выполнить проектирование элементов автоматизированной системы управления (АСУ) в соответствии с темой курсовой работы.

Задачи курсовой работы:
С помощью технологии OSINT осуществить сбор информации об организации по направлению деятельности в соответствии с темой курсовой работы.
Разработать функциональные и нефункциональные требования к АСУ в соответствии с темой курсовой работы с использованием реверс-инжиниринга.
Выполнить моделирование АСУ в соответствии с темой курсовой работы с использованием средств визуализации стандарта UML с разбиением АСУ на модули (обязательное представление диаграмм вариантов использования для каждого модуля, диаграммы последовательности для основного процесс деятельности организации и диаграммы классов).
Выполнить выбор и описание программно-аппаратной платформы, сформировать перечень программного обеспечения для обеспечения всех функций АСУ, указанных в требованиях и отображенных на UML диаграммах.
Определить функцию АСУ, которая в наименьшей степени обеспечивается выбранным программным обеспечением. Выбрать IDE, библиотеки для создания программного модуля для проектируемой АСУ, реализующего данную функцию. Представить исходный код

разработанного модуля.

Построить ER-диаграмму на основе классовой диаграммы UML, выбрать систему управления базами (СУБД) данных и реализовать схему данных в выбранной СУБД.

Выбрать серверное и клиентское оборудование для обеспечения выполнения функций, отображенных на UML диаграммах, построить схему сети с использованием средств виртуализации. Выбрать среду виртуализации, создать виртуальный стенд, реализующий виртуальную инфраструктуру АСУ.

Создать программный модуль с Web-интерфейсом для работы с базой данных АСУ.

Предложить решение по обеспечению безопасности для АСУ, выбрав средства обеспечения безопасности обработки и передачи информации в соответствии со схемой сети.

Требования к курсовой работе:

По результатам выполнения курсовой работы формируется отчет. Выполнение каждой задачи курсовой работы подтверждается разделом отчета с обязательными выводами о проделанной работе. Все результаты выполнения задач курсовой работы, представленные в отчете, должны быть получены лично автором, все заимствования оформляются со ссылкой на авторитетный источник, указанный в списке использованной литературы в конце отчета.

Срок представления к защите курсовой работы: **до «25» 05 2026 г.**

Задание на курсовую работу выдал _____ Щербаков К.В.

Подпись руководителя *(ФИО руководителя)*

«12» 02 2026 г.

Задание на курсовую работу получил _____ Астахов С.П.

Подпись обучающегося *(ФИО обучающегося)*

«10» 02 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 7

Структура и описание проектируемой АСУ. 10

1. Общая характеристика организации 10

2. Логическая структура АСУ 11

3. Функциональные задачи системы 11

4. Должностные лица(пользователи системы) 12

5. Жизненный цикл проектирования 12

6. Нормативные-правовые акты 13

Функциональные и нефункциональные требования к АСУ 15

1. Функциональные требования: 15

2. Нефункциональные требования: 17

UML моделирование проектируемой АСУ. 21

1. Модули АСУ РТОП, интерфейсные и внутрисистемные функции 22

2. Диаграммы последовательности 28

3. Диаграмма классов проектируемой АСУ 31

Описание и доработка программно-аппаратной платформы для проектируемой АСУ. 33

1. Таблица аппаратного обеспечения АСУ по выбранной тематике и обеспечивающих подсистем 34

2. Таблица программного обеспечения для выполнения всех функций, описанных в требованиях и отображенных на UML диаграммах 38

3. Анализ существующих языков программирования и IDE 41

4. Языки программирования 41

5. Интегрированные среды разработки (IDE) 42

6. Обоснованный выбор инструментов 44

7. Выбор и описание библиотек 45
8. Реализация 45
ER-диаграмма для проектируемой АСУ. 53
1. Выбор СУБД 53
2. Описание ER-диаграммы АСУ поддержание температуры в оранжерее55
Схема сети для проектируемой АСУ 68
1. Клиентское оборудование 68
2. Серверное оборудование 69
2.1. Сервер баз данных 69
2.2 Серверы приложений (SCADA / Веб-сервер / MQTT Broker) 69
2.3 Промышленные контроллеры (PLC / Edge-устройства) 70
2.4 Серверы резервного копирования и мониторинга 71
2.5 Диспетчерская рабочая станция(визуализация и управление) 71
3. Сетевое оборудование 72
4. Сравнение сред виртуализации и обоснование выбора 73
4.1. Критерии сравнения 73
4.2. Анализ преимуществ и недостатков 74
5. Обоснование выбора среды виртуализации 74
6. Проектирование виртуального стенда 75
6.1 Архитектура стенда 75
6.2 Распределение ПО по виртуальным машинам 76
6.3. Аппаратные ресурсы хоста и распределение 76
Web-интерфейс для работы с базой данных АСУ 78
1. Назначение Web-интерфейса 78
2. Структура Web-интерфейса 79

3. Описание страниц Web-интерфейса 79
4. Архитектура Web-модуля 83
5. Листинг исходного кода Web-интерфейса 83
Средства защиты информации для проектируемой АСУ 94
ЗАКЛЮЧЕНИЕ 99
СПИСОК ЛИТЕРАТУРЫ 102

ВВЕДЕНИЕ

В современных условиях обеспечение стабильного урожая в тепличных хозяйствах напрямую связано с надёжной работой систем поддержания микроклимата. Средства обогрева, вентиляции, увлажнения и затенения должны функционировать непрерывно, обеспечивая агрономические службы достоверной информацией о температуре, влажности, концентрации CO_2 и состоянии исполнительных механизмов. При этом эксплуатация большого количества разнотипных датчиков и устройств в многозонной оранжерее требует централизованного контроля, своевременной диагностики, регистрации событий и поддержки принятия решений агрономом и инженерно-техническим персоналом.

Актуальность темы курсовой работы: необходимость повышения надёжности поддержания микроклимата в оранжерее, сокращения времени реакции на отклонения температурных параметров, снижения влияния человеческого фактора и обеспечения единого информационного пространства для агронома, инженера-технолога, начальника тепличного цеха и системного администратора. Использование автоматизированной системы управления позволяет объединить данные от датчиков температуры, влажности и CO_2, обеспечить их обработку, хранение, визуализацию и защиту.

Объект автоматизации: теплица (оранжерея), предназначенная для круглогодичного выращивания овощных, ягодных и цветочных культур. Автоматизации подлежит участок деятельности, связанный с мониторингом параметров микроклимата, управлением режимами работы обогревателей, вентиляции и затенения, обработкой аварийных ситуаций, ведением электронного архива показаний и формированием отчётности.

Предмет: проектирование автоматизированной системы управления поддержанием температуры в оранжерее — АСУ ТП «Микроклимат». Проектируемая система относится к локальным диспетчерским человеко-машинным системам, работающим в режиме реального времени, и предназначена для централизованного контроля температурных режимов, сбора и обработки

телеметрии с датчиков, формирования аварийных сообщений, архивирования данных и поддержки эксплуатационного персонала.

Цель курсовой работы: проектирование элементов автоматизированной системы управления поддержанием температуры в оранжерее в соответствии с требованиями к АСУ промышленного и административного объекта.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Выполнить анализ предметной области и описать структуру проектируемой АСУ, включая бизнес-процессы, логическую структуру, основные функции и пользователей системы.
2. Сформировать функциональные и нефункциональные требования к АСУ ТП «Микроклимат».
3. Выполнить UML-моделирование проектируемой системы, включая диаграммы вариантов использования, диаграммы последовательности и диаграмму классов.
4. Описать программно-аппаратную платформу АСУ, выбрать программное обеспечение и разработать программный модуль для функции, требующей доработки (автоматическая настройка ПИД-регулятора).
5. Построить ER-диаграмму на основе UML-модели, выбрать СУБД и реализовать структуру базы данных.
6. Разработать схему сети для проектируемой АСУ и описать виртуальную инфраструктуру.
7. Создать веб-интерфейс для работы с базой данных АСУ.
8. Предложить средства защиты информации, соответствующие структуре сети и характеру обрабатываемых данных.

В ходе выполнения курсовой работы разрабатывается комплексное проектное решение, включающее описание структуры АСУ, требования к системе, UML-модели, программно-аппаратную платформу, базу данных, сетевую инфраструктуру, веб-интерфейс и средства защиты информации.

Результат работы: проект автоматизированной системы управления, направленной на повышение эффективности, надёжности и безопасности поддержания микроклимата в оранжерее, что обеспечивает стабильный рост растений и сохранность урожая.

Структура и описание проектируемой АСУ.

Цель раздела:

Выполнить сбор и анализ открытой информации об организации и предметной области, связанной с автоматизированным управлением микроклиматом в тепличных хозяйствах, определить автоматизируемый участок деятельности, выделить основные бизнес-процессы, пользователей, функции и логическую структуру проектируемой АСУ.

1. Общая характеристика организации

Наименование проектируемой системы: Автоматизированная система управления технологическим процессом «поддержание температуры в оранжереи»

Направление деятельности организации: Поддержание комфортной температуры растений в теплице и регулирование температуры по предпочтению пользователя. Система может быть как автоматизированной, так и ручной, в зависимости от использования

Анализ текущей ситуации (AS-IS):

В настоящее время теплицы корректируют все время закрытым утепленным помещением, тем самым температура там все время одна и таже. Это приводит к следующим проблемам:

- Нет контроля температуры для микроклимата;
- Нет возможности менять температуры для определенных сортов растений;
- Невозможность корректировки роста растений;
- Нет обработки изображений растений для определения этапов роста.

Процесс, требующий автоматизации: процесс поддержание температуры в умной теплице.

Ключевое нововведение: вместо ручного труда, теплица сама регулирует параметры поддержание в зависимости от введённых входных данных.

Автоматизации подлежат:

- Процесс измерения текущей температуры воздуха и почвы в различных зонах оранжереи;

- Процесс сравнения измеренных значений с заданными уставками (в зависимости от типа растений и стадии их роста);
- Процесс автоматического включения/отключения исполнительных механизмов (нагреватели, вентиляция, затенение, форточки);
- Процесс визуального контроля состояния растений с помощью камер для определения этапов роста;
- Процесс регистрации и архивирования параметров микроклимата;
- Процесс удаленного мониторинга и управления через интерфейс оператора.

Тип проектируемой АСУ: Проектируемая система относится к классу АСУ ТП (автоматизированная система управления технологическим процессом). По функциональному назначению — система управления режимами (температурой)

2. Логическая структура АСУ

Система состоит из следующих функциональных подсистем:

1. Подсистема измерения и контроля — сбор данных с датчиков температуры воздуха, температуры почвы, влажности, CO_2, освещенности; передача данных на контроллер.

2. Подсистема управления исполнительными механизмами — автоматическое включение/отключение нагревателей, вентиляторов, приводов открытия форточек, системы затенения.

3. Подсистема технического зрения — обработка изображений с камер для определения стадии роста растений и выявления визуальных отклонений.

4. Подсистема хранения и архивирования — база данных для хранения показаний датчиков, событий, действий оператора и архивных записей.

5. Подсистема человеко-машинного интерфейса (SCADA/HMI) — отображение текущих параметров, графиков, настройка уставок, вывод тревог, удаленный доступ.

3. Функциональные задачи системы

1. Автоматический сбор данных — опрос датчиков с заданной периодичностью.
2. ПИД-регулирование температуры — поддержание заданной температуры с

минимальными отклонениями.

3. Управление по расписанию — автоматическая смена дневных и ночных режимов.
4. Обработка изображений — распознавание стадий роста растений с помощью нейросети.
5. Регистрация аварийных событий — фиксация превышений допустимых пределов, отправка уведомлений оператору.
6. Ведение архива — хранение данных за весь период эксплуатации.
7. Удаленный мониторинг — доступ к системе через веб-интерфейс.

4. Должностные лица(пользователи системы)

1. Пользователь:
   - просмотр текущих параметров микроклимата;
   - настройка температурных уставок в зависимости от типа растений и стадии роста;
   - переход между автоматическим и ручным режимом управления;
   - просмотр архива и графиков изменений параметров;
   - получение уведомлений об отклонениях и авариях.
2. Администратор системы:
   - настройка прав доступа пользователей;
   - обслуживание оборудования (датчиков, контроллеров, исполнительных механизмов);
   - резервное копирование базы данных;
   - обновление программного обеспечения;
   - настройка интеграции с внешними системами (метеостанция, API).

5. Жизненный цикл проектирования

Жизненный цикл состоит из следующих частей[3]:

1. Разработка концепции – создание моделей бизнес-процессов и функциональной структуры.

2. Разработка концепции – создание моделей бизнес-процессов и

функциональной структуры.

3. Техническое задание – детализация требований к системе(архитектура), интеграции и др.

4. Технический проект – разработка архитектуры базы данных, проектирование API.

5. Рабочая документация – кодирование, создание интерфейсов.

6. Ввод в действие – тестирование на реальной эксплуатации, обучение пользователей.

7. Сопровождение – добавление новых источников контента.

6. Нормативные-правовые акты

- ГОСТ 34.601-90 — «Автоматизированные системы. Стадии создания»[3];
- ГОСТ 34.201-2020 — «Виды, комплектность и обозначение документов при создании АСУ» [1];
- ГОСТ 34.602-2020 — «Техническое задание на создание автоматизированной системы»[2];
- ГОСТ Р 55061-2012 — «Автоматизированные системы управления технологическими процессами. Термины и определения»;
- Федеральный закон № 384-ФЗ «Технический регламент о безопасности зданий и сооружений» (в части обеспечения микроклимата);
- Правила технической эксплуатации электроустановок потребителей (ПТЭЭП) для безопасной эксплуатации нагревательного оборудования.

Вывод по разделу:

В результате выполненной раздела был проведен анализ предметной области автоматизации процесса поддержания температуры в оранжерее. Выявлены недостатки существующего подхода (AS-IS), связанные с отсутствием гибкости управления и контроля микроклимата. Обоснована необходимость внедрения автоматизированной системы управления технологическим процессом (АСУ ТП).

Разработана концепция системы (TO-BE), включающая автоматический сбор данных с датчиков, управление исполнительными механизмами, обработку изображений с камер для определения этапов роста растений, а также удаленный мониторинг и управление. Определены функциональные подсистемы, пользователи системы и жизненный цикл проектирования в соответствии с ГОСТ.

Функциональные и нефункциональные требования к АСУ.

Цель раздела:

Сформировать функциональные и нефункциональные требования к проектируемой АСУ на основе анализа предметной области, открытых источников, нормативных требований и принципов реверс-инжиниринга существующих автоматизированных систем.

Введение:

В данном разделе сформированы функциональные и нефункциональные требования к проектируемой автоматизированной системе управления поддержанием температуры в оранжерее. Требования определены на основе анализа предметной области, нормативных требований к системам автоматизации тепличных хозяйств, а также с использованием подхода реверс-инжиниринга существующих автоматизированных систем контроля и управления микроклиматом в агропромышленных комплексах.

1. Функциональные требования:

№	Функциональные требования	Источник
1	Система должна обеспечивать автоматический сбор данных с датчиков температуры воздуха (не менее 2 датчиков на отделение)	SCADA TRACE MODE (внедрена на Агрокомбинате «Московский»)
2	Система должна измерять влажность воздуха в каждой климатической зоне оранжереи	SCADA TRACE MODE (внедрена на Агрокомбинате «Московский»)
3	Система должна получать данные с локальной метеостанции (температура наружного воздуха, направление и скорость ветра)	SCADA TRACE MODE (внедрена на Агрокомбинате «Московский»)
4	Система должна измерять температуру теплоносителя на входе и выходе из каждого отделения	SCADA TRACE MODE (внедрена на Агрокомбинате «Московский»)
5	Система должна измерять давление воды системы отопления на входе и выходе	SCADA TRACE MODE (внедрена на Агрокомбинате «Московский»)

6	Система должна управлять системами обогрева (подпочвенный, надпочвенный, зональный, кровельный)	"Умная теплица" (НПО "АвтоматикА")
7	Система должна управлять форточками (фрамугами) с учётом направления и скорости ветра	"Умная теплица" (НПО "АвтоматикА")
8	Система должна обеспечивать ПИД-регулирование температуры воздуха	SCADA TRACE MODE (внедрена на Агрокомбинате «Московский»)
9	Система должна включать системы обогрева в иерархическом порядке (надпочвенный → зональный → кровельный при недостаточности предыдущего)	"Умная теплица" (НПО "АвтоматикА")
10	Система должна управлять рециркуляционными вентиляторами для обеспечения равномерного распределения тепла	"Умная теплица" (НПО "АвтоматикА")
11	Система должна управлять системой зашторивания (затенения) для регулировки солнечной радиации	"Умная теплица" (НПО "АвтоматикА")
12	Система должна управлять электродосвечиванием растений	"Умная теплица" (НПО "АвтоматикА")
13	Система должна управлять системой CO_2 для интенсификации фотосинтеза	SCADA TRACE MODE (внедрена на Агрокомбинате «Московский»)
14	Система должна обеспечивать управление поливом растений	"Умная теплица" (НПО "АвтоматикА")
15	Система должна архивировать значения температуры воздуха за последние 24 часа в графической форме	"Умная теплица" (НПО "АвтоматикА")
16	Система должна защищать систему подпочвенного обогрева от перегрева (автоматическое отключение насоса и закрытие клапана при превышении заданной температуры на 10°C)	"Умная теплица" (НПО "АвтоматикА")
17	Система должна обеспечивать дискретное управление задвижками отопления и фрамугами для каждого отделения	SCADA TRACE MODE (внедрена на Агрокомбинате «Московский»)

18	Система должна регистрировать и сохранять суммарную солнечную радиацию (для анализа фотосинтеза)	SCADA TRACE MODE (внедрена на Агрокомбинате «Московский»)
19	Система должна обеспечивать возможность накопления статистики с выдачей рекомендаций по режимам выращивания	SCADA TRACE MODE (внедрена на Агрокомбинате «Московский»)
20	Система должна управлять тепловым узлом (регулирование температуры теплоносителя для систем обогрева)	"Умная теплица" (НПО "АвтоматикА")

2. Нефункциональные требования:

№	Нефункциональные требования	Источник
1	SCADA-система должна быть способна работать на отечественном программном обеспечении (TRACE MODE или аналог) в рамках политики импортозамещения	Постановление Правительства РФ № 325-ФЗ "О промышленной политике", ФЗ № 188-ФЗ "Об импортозамещении"
2	Система должна соответствовать требованиям Ростехнадзора для объектов, подконтрольных надзорным органам	Нормативные документы Ростехнадзора по построению систем технологических защит и защитных блокировок
3	Контроллеры должны быть совместимы со стандартами IEC 61131-3 и IEC 614999 для программирования (языки: LD, FBD, ST, IL, SFC)	Нормативные документы Ростехнадзора по построению систем технологических защит и защитных блокировок
4	Система должна обеспечивать передачу данных в СУБД через интерфейс ODBC для документирования процесса	ГОСТ Р МЭК 61131-3-2016 "Контроллеры программируемые. Языки программирования"
5	Аппаратная часть должна быть адаптирована к российским условиям эксплуатации (диапазон температур для неотапливаемых помещений от -10°C до +50°C)	ГОСТ Р 52931-2008 "Приборы контроля и регулирования технологических процессов. Общие технические условия"
6	Оборудование УСО (устройств связи с объектом) должно быть отечественного производства для ответственных применений	Стандарты организаций по построению АСУ ТП с учетом политики импортозамещения в промышленности и энергетике

7	SCADA-система должна иметь лицензию на необходимое количество точек ввода-вывода (не менее 32000 для масштабирования)	Технические требования к АСУ ТП крупных агрокомбинатов (12-й блок АК "Московский" — 32000 точек)
8	Система должна иметь простой и интуитивно понятный интерфейс для оперативного персонала с минимальным временем обучения	ГОСТ 34.602-2020 "Техническое задание на создание автоматизированной системы" (раздел "Требования к эргономике")
9	Система должна обеспечивать время реакции на отклонение параметров не более заданного порога (отклонение температуры для форточек — настраиваемая дельта)	ГОСТ Р МЭК 60870-5-104-2004 "Устройства и системы телемеханики" (требования к времени доставки сообщений)
10	Система должна быть масштабируемой: от управления одним блоком до корпоративной системы всего агрокомбината	ГОСТ Р 55061-2012 "Автоматизированные системы управления технологическими процессами. Масштабируемость"
11	Контроллеры должны поддерживать работу с распределенными модулями ввода-вывода серии Advantech 5000 или отечественными аналогами	Требования к совместимости оборудования АСУ ТП (опыт АК "Московский" с распределенными УСО)
12	Система должна обеспечивать возможность тест-драйва перед внедрением для проверки функциональности	ГОСТ 34.603-2020 "Виды испытаний автоматизированных систем" (предпусковая отработка)
13	Время выполнения модернизации системы (реконструкции) не должно превышать 2 рабочих дней для критических объектов	Опыт реконструкции "Умной теплицы" в Сысерти (НПО "АвтоматикА" за 2 дня)
14	Система должна поддерживать формирование отчетов о параметрах микроклимата с помощью специализированного генератора отчетов	ГОСТ Р ИСО/МЭК 25051-2017 "Требования к качеству программного продукта"
15	Точность поддержания температуры воздуха должна позволять реализацию научных методов выращивания растений	Требования агрономической службы Агрокомбината

		"Московский" к качеству температурного контроля
16	Система должна быть гибкой и легко адаптироваться к новому оборудованию и изменениям технологии выращивания	ГОСТ 34.601-2020 "Автоматизированные системы. Стадии создания" (требования к модифицируемости)
17	Система должна стоить не дороже импортных аналогов при сопоставимой функциональности	Технико-экономическое обоснование при выборе SCADA-системы (TRACE MODE vs зарубежные климат-компьютеры)
18	Система должна обеспечивать согласованную работу систем обогрева и вентиляции (исключение их одновременной работы в штатном режиме)	Алгоритмы согласованного управления (опыт НПО "АвтоматикА" с дельтой температуры для форточек)
19	Система должна обеспечивать защиту от несанкционированного доступа к настройкам и архивным данным	ГОСТ Р 56545-2015 "Защита информации. Правила разработки безопасного ПО"
20	Период хранения архива параметров микроклимата должен позволять анализ условий выращивания за длительный срок (не менее 1 года)	Агрокомбинат "Московский": возможность просмотра параметров "день назад, неделю, месяц" с накоплением статистики

Вывод по разделу:

В результате выполнения раздела были сформированы функциональные и нефункциональные требования к проектируемой АСУ поддержания температуры в оранжерее.

Функциональные требования определяют состав основных возможностей системы: автоматический сбор показаний с датчиков температуры, влажности и CO_2, управление исполнительными механизмами (обогреватели, вентиляция, затенение, форточки), ПИД-регулирование температуры, обработку изображений для определения стадий роста растений, регистрацию аварийных событий, ведение

архива параметров микроклимата, формирование отчётности и разграничение доступа пользователей.

Нефункциональные требования определяют показатели надёжности (бесперебойная работа 24/7), производительности (обработка показаний от 100 датчиков с интервалом 5-10 секунд), отказоустойчивости (резервирование контроллеров и каналов связи), безопасности (защита от несанкционированного доступа, антивирусная защита, шифрование данных), масштабируемости (возможность добавления новых зон оранжереи и типов датчиков) и условий эксплуатации (диапазон температур от -10°C до +50°C для полевого оборудования).

Сформированные требования являются основой для дальнейшего UML-моделирования, выбора программно-аппаратной платформы, проектирования базы данных, сети и средств защиты информации для АСУ поддержания температуры в оранжерее.

UML моделирование проектируемой АСУ.

Цель раздела:

Выполнить моделирование проектируемой АСУ поддержание температуры в оранжерее с использованием средств визуализации стандарта UML, определить структуру системы по модулям, описать взаимодействие пользователей с системой, основные процессы работы и статическую структуру классов.

Введение:

В данном разделе выполнено UML-моделирование проектируемой автоматизированной системы управления поддержанием температуры в оранжерее. Моделирование выполнено с разбиением АСУ на функциональные модули. Для каждого модуля определены пользователи, интерфейсные функции и внутрисистемные функции, а также построены UML-диаграммы вариантов использования.

На основе функциональных требований, выделены три основных модуля АСУ ТП «Регулировка температуры в оранжерее»:

- Модуль «Мониторинг и визуализации параметров»
- Модуль «Управление исполнительными механизмами»
- Модуль «Администрирование, логирование и отчетность»

В проектируемой АСУ поддержанию температуры в оранжереи выделены следующие категории пользователей:

Таблица 1 - Должностные лица — пользователи АСУ поддержанию температуры в оранжерее — и их функциональные обязанности

№	Должность	Функциональные обязанности
1	Оператор / агроном	Мониторинг температуры, влажности, солнечной радиации; ручное управление обогревом/форточками; просмотр трендов
2	Начальник цеха	Контроль KPIs, утверждение уставок, просмотр отчетов по энергопотреблению и климату, анализ аварий
3	Инженер-технолог (АСУ ТП)	Настройка ПИД-регуляторов, изменение дельт для форточек, калибровка датчиков, управление логикой контроллеров

4	Администратор SCADA	Управление пользователями и правами, резервное копирование, настройка архивов, лицензирование
5	Обслуживающий персонал (КИПиА)	Просмотр состояния оборудования, диагностика отказов, ручное тестирование задвижек и насосов

1. Модули АСУ РТОП, интерфейсные и внутрисистемные функции

В составе проектируемой АСУ поддержанию температуры в оранжерее выделены три модуля. Для каждого модуля сформированы: матрица пользователей, внутрисистемные функции и use case диаграмма.

Модуль 1: «Мониторинг и визуализация»

Назначение:

Сбор, отображение и архивирование данных с датчиков температуры, влажности, метеостанции, теплоносителя, давления.

Возможности модуля: графики трендов, мнемосхемы отделений, экстренные индикаторы, просмотр архивов за 24 часа.

Таблица 2 — Матрица пользователей Модуля 1

ID	Интерфейсная функция	Оператор	Нач. цеха	Инженер	Админ SCADA	КИПиА
1.1	Просмотр текущей температуры (по отделениям)	X	X	X		X
1.2	Просмотр влажности воздуха	X				
1.3	Просмотр данных метеостанции	X	X	X		
1.4	График температуры за 24 часа	X	X	X		X
1.5	Просмотр суммарной солнечной радиации	X				
1.6	Мнемосхема оранжереи с индикацией клапанов	X		X		X
1.7	Просмотр температуры теплоносителя на входе/выходе	X		X		X
1.8	Просмотр давления воды в системе отопления	X				

1.9	Аварийный дашборд (отклонения >10°C и т.д.)	X	X	X		X
1.10	Просмотр статуса связи с контроллерами			X	X	X

Таблица 3 — Матрица внутрисистемные функций Модуля 1

ID интерф. функции	Интерфейсная функция	Внутрисистемная функция (триггер)
1.1	Просмотр текущей температуры по отделениям	Циклический OPC-запрос к ПЛК (частота 1 Гц)
1.2	Просмотр влажности воздуха	Запрос к аналоговым входам модуля УСО
1.3	Просмотр данных метеостанции	Чтение регистров по Modbus TCP от метеостанции
1.4	График температуры за 24 часа	Выборка из архива historian со сжатием данных
1.5	Просмотр суммарной солнечной радиации	Запрос к пиранометру через аналоговый вход
1.6	Мнемосхема оранжереи с индикацией клапанов	Подписка на тэги OPC с автоматическим обновлением экрана
1.7	Просмотр температуры теплоносителя (вход/выход)	Чтение с датчиков DS18B20 через 1-Wire интерфейс
1.8	Просмотр давления воды в системе отопления	Преобразование токового сигнала 4-20 мА в физическое значение
1.9	Аварийный дашборд (отклонения >10°C)	Фоновая задача сравнения T > T_уставка + 10°C
1.10	Просмотр статуса связи с контроллерами	Периодическая отправка KeepAlive и проверка ответа

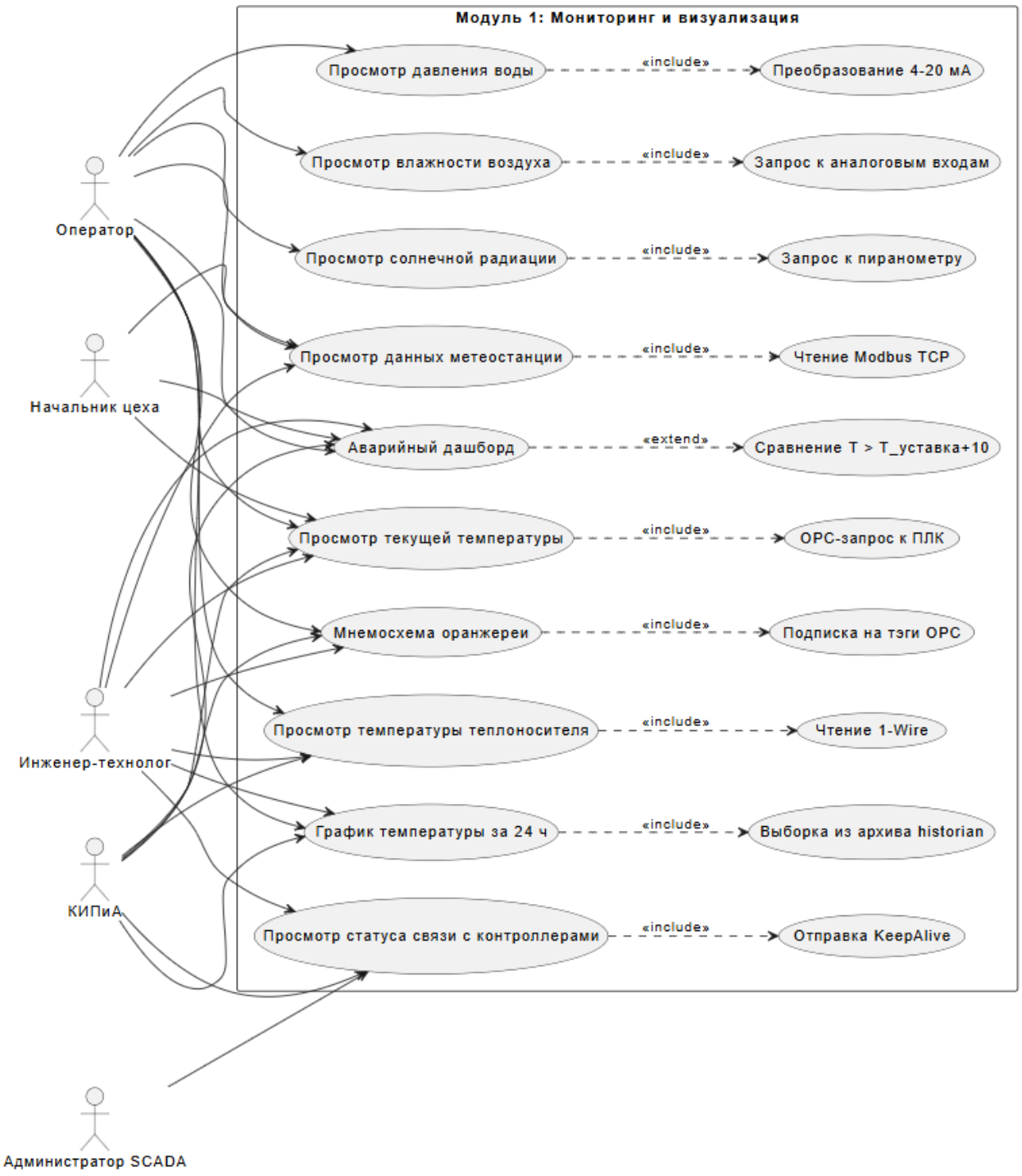


Рисунок 1 – Use Case диаграмма Модуля 1

Модуль 2: «Управление исполнительными механизмами»

Назначение:

Автоматическое и ручное управление системами обогрева, форточками, вентиляторами, зашториванием, досвечиванием, поливом, CO_2.

Возможности модуля: ПИД-регулирование, иерархическое включение обогрева, защита от перегрева, управление тепловым узлом.

Таблица 4 — Матрица пользователей и интерфейсных функций Модуля 2

ID	Интерфейсная функция	Оператор	Нач. цеха	Инженер	Админ SCADA	КИПиА
2.1	Ручное открытие/закрытие фрамуги (форточки)	X				X
2.2	Задание уставки температуры для ПИД-регулятора	X	X	X		
2.3	Включение/выключение подпочвенного обогрева	X				
2.4	Иерархический запуск обогрева (надпочвенный→зональный→кровельный)	X		X		
2.5	Управление рециркуляционными вентиляторами	X				
2.6	Управление зашториванием	X				
2.7	Управление электродосвечиванием	X				
2.8	Управление подачей CO_2	X				
2.9	Управление тепловым узлом			X		X
2.10	Защитное отключение подпочвенного обогрева (при перегреве)	X		X		X

Таблица 5 — Внутрисистемные функции Модуля 2

ID интерф. функции	Интерфейсная функция	Внутрисистемная функция (триггер)
2.1	Ручное открытие/закрытие фрамуги	Отправка команды DO на контроллер + проверка концевика
2.2	Задание уставки температуры для ПИД	Обновление переменной SP в ПЛК, пересчёт управляющего сигнала CV
2.3	Включение/выключение подпочвенного обогрева	Команда на модуль реле + контроль обратной связи по току
2.4	Иерархический запуск обогрева	Цикличная проверка: надпочвенный → зональный → кровельный
2.5	Управление рециркуляционными вентиляторами	Формирование ШИМ-сигнала или дискретное включение

2.6	Управление зашториванием	Команда на сервопривод + считывание энкодера положения
2.7	Управление электродосвечиванием	Релейное включение LED-фитоламп по таймеру или радиации
2.8	Управление подачей CO_2	Открытие/закрытие электромагнитного клапана подачи газа
2.9	Управление тепловым узлом	Позиционирование трёхходового клапана по заданному углу
2.10	Защитное отключение подпочвенного обогрева	Аппаратное прерывание + блокировка дискретного выхода + запись в лог

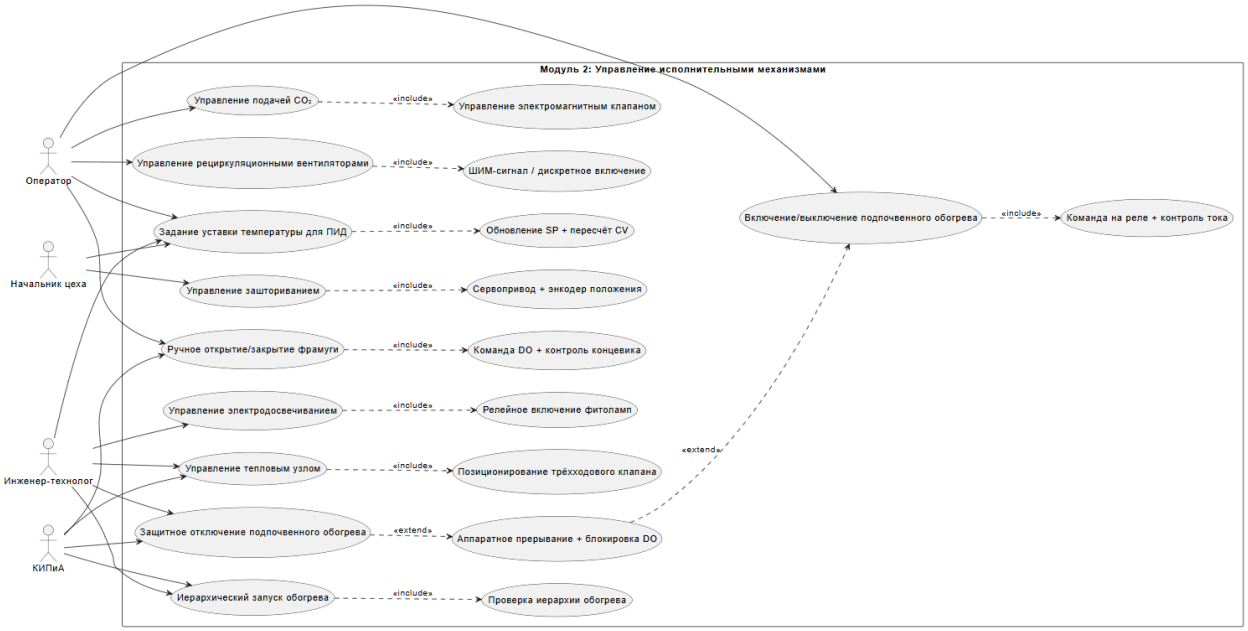


Рисунок 2 – Use Case диаграмма модуля 2

Модуль 3: «Администрирование, логирование, отчеты»

Назначение:

Настройка системы, контроль доступа, регистрация аварий, формирование отчетов для агронома и начальника цеха.

Возможности модуля:

Управление пользователями/ролями, просмотр логов, генерация отчетов по микроклимату, настройка параметров ПИД и дельт.

Таблица 6 — Матрица пользователей и интерфейсных функций Модуля 3

ID	Интерфейсная функция	Оператор	Нач. цеха	Инженер	Админ SCADA	КИПиА
3.1	Просмотр лога событий и аварий	X	X	X	X	X
3.2	Формирование отчета по параметрам микроклимата		X	X		
3.3	Настройка дельты для автоматики форточек			X		
3.4	Настройка ПИД-коэффициентов (Kp, Ki, Kd)			X		
3.5	Управление пользователями и ролями				X	
3.6	Настройка архивов (период хранения 1 год)				X	
3.7	Просмотр лицензии SCADA и количества точек ввода-вывода				X	
3.8	Тест-драйв (симуляция) перед внедрением			X	X	X
3.9	Конфигурация ODBC-экспорта в СУБД				X	
3.10	Просмотр отчета по отклонениям температуры за смену	X	X			

Таблица 7 — Внутрисистемные функции Модуля 3

ID интерф. функции	Интерфейсная функция	Внутрисистемная функция (триггер)
3.1	Просмотр лога событий и аварий	SQL-запрос к таблице системных событий с фильтрацией по времени
3.2	Формирование отчёта по параметрам микроклимата	Агрегация данных из historian за выбранный период
3.3	Настройка дельты для автоматики форточек	Запись в конфигурационный файл контроллера + перезапуск задачи
3.4	Настройка ПИД-коэффициентов (Kp, Ki, Kd)	Передача новых параметров в ПЛК через OPC-интерфейс
3.5	Управление пользователями и ролями	Выполнение UPDATE прав доступа в таблице БД SCADA
3.6	Настройка архивов (период хранения)	Изменение политики ротации логов и historian

3.7	Просмотр лицензии SCADA и количества точек	Чтение лицензионного файла и подсчёт активных тэгов
3.8	Тест-драйв (симуляция) перед внедрением	Переключение УСО на симулятор и отключение реальных I/O
3.9	Конфигурация ODBC-экспорта в СУБД	Проверка соединения с БД и настройка строки подключения
3.10	Резервное копирование конфигурации	Копирование папки проекта в архив с меткой времени

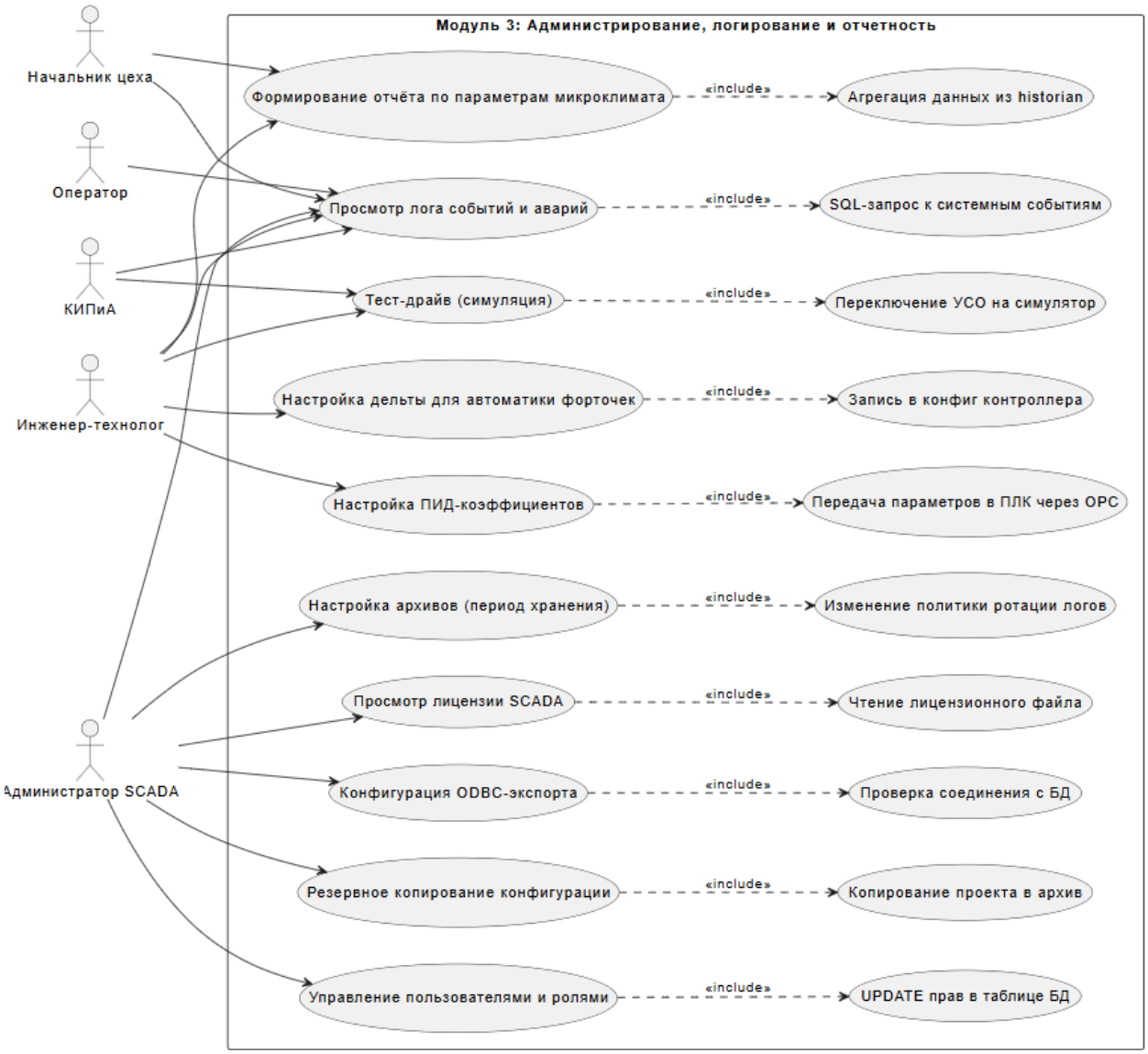


Рисунок 3 – Use Case диаграмма Модуля 3

2. Диаграммы последовательности

ПИД-регулирование

В этой диаграмме участвуют: Оператор (наблюдает), Инженер-технолог (косвенно), КИПиА (диагностика).

Начальник цеха и Администратор в этом процессе не участвуют.

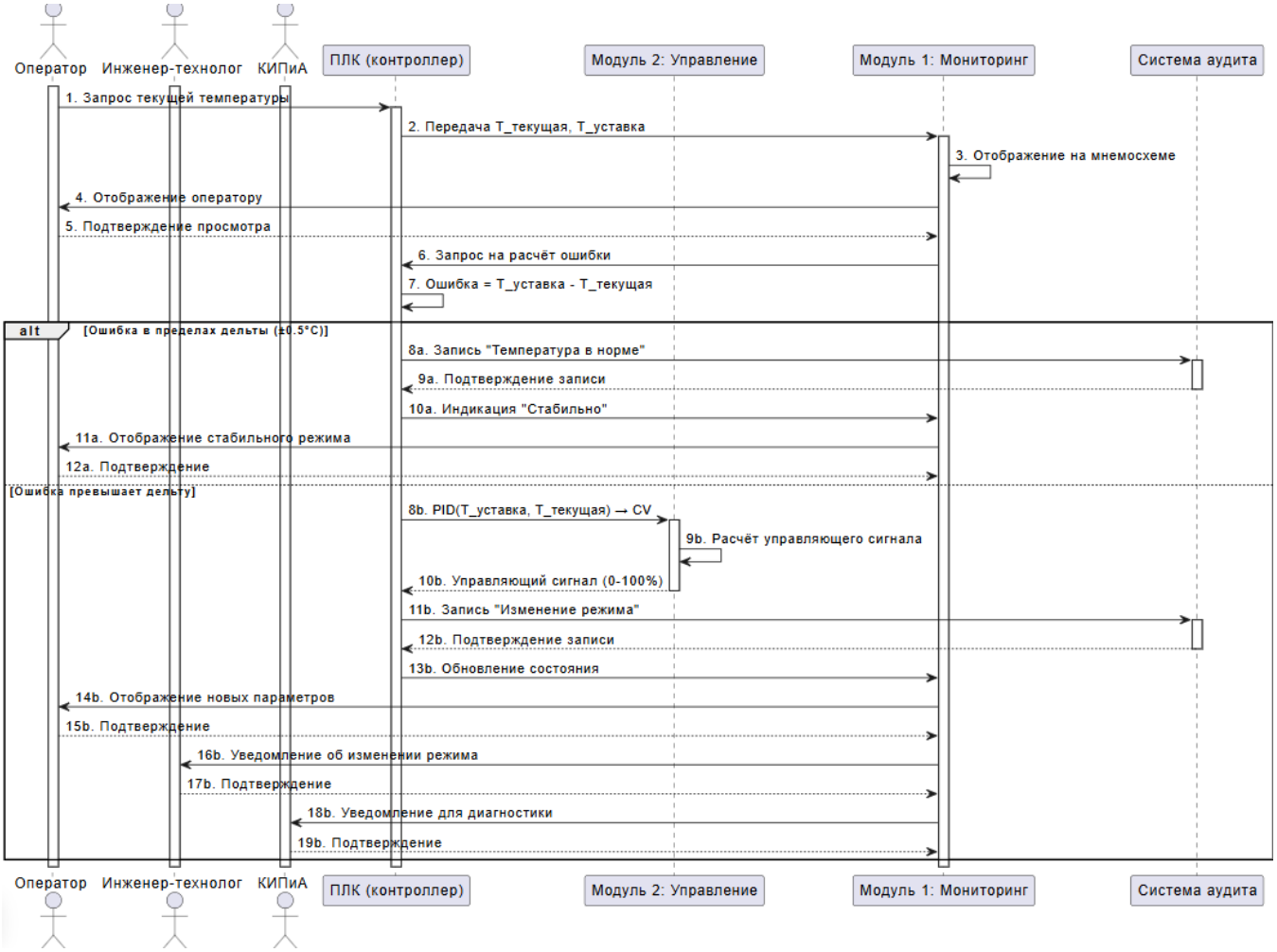


Рисунок 4 —ПИД-регулирование

Защитное отключение при перегреве

Участвуют: Инженертехнолог, КИПиА, Оператор (наблюдает), Администратор (получает лог). Начальник цеха — только в отчётах, в этом процессе не участвует.

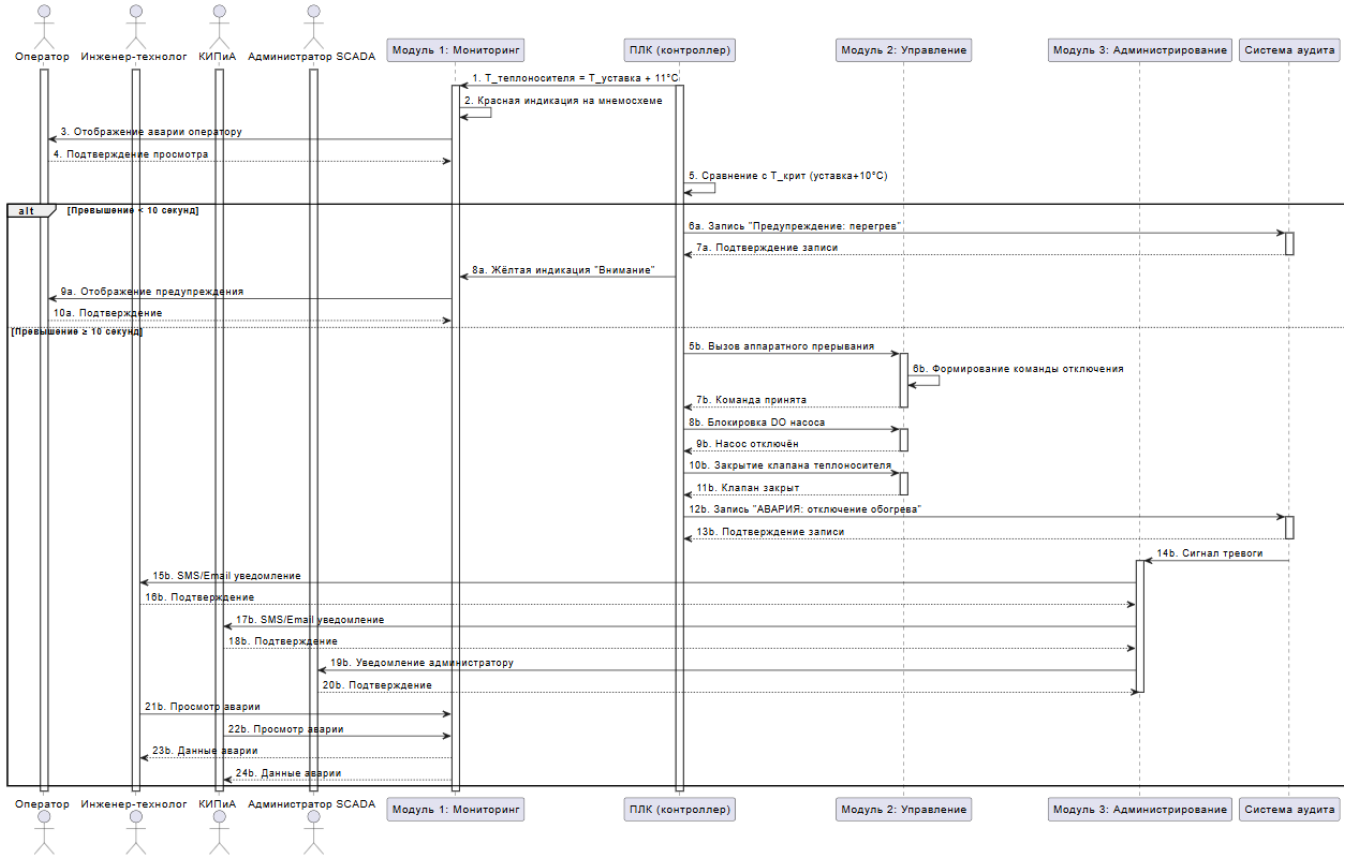


Рисунок 5 — Защитное отключение при перегреве

Настройка ПИД-коэффициентов

Участвуют: Инженертехнолог (основной), Администратор (лог), Оператор (н аблюдает эффект). КИПиА и Начальник цеха в этом процессе не участвуют.

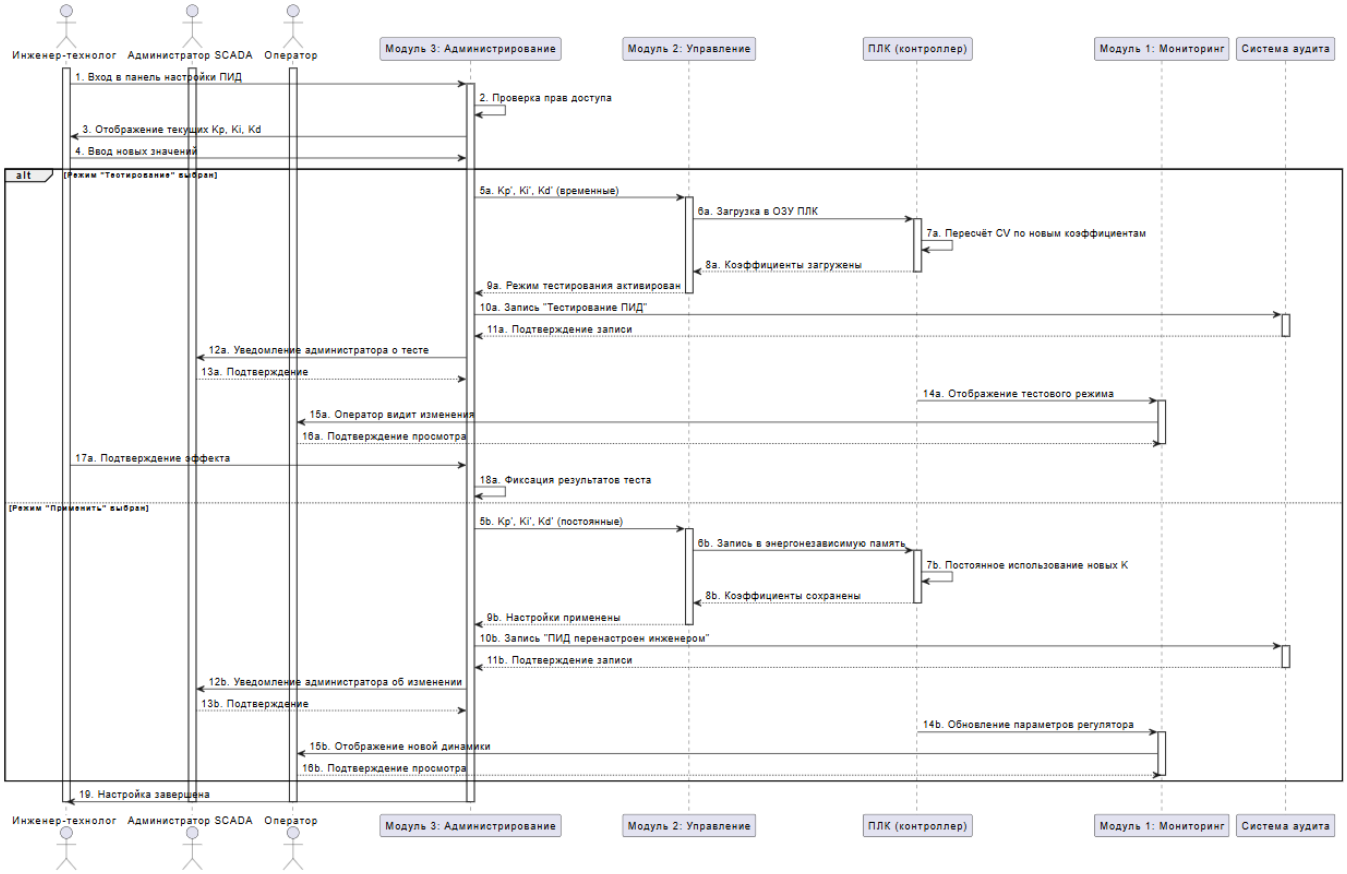


Рисунок 6 — Настройка ПИД-коэффициентов

3. Диаграмма классов проектируемой АСУ

Диаграмма классов отражает статическую структуру проектируемой АСУ ТП поддержания температуры в оранжерее. На ней представлены основные классы системы, их атрибуты, методы и связи. Центральным элементом диаграммы является класс ClimateZone (Климатическая зона), который выполняет роль управляемого объекта и обеспечивает взаимодействие с датчиками температуры и влажности, исполнительными механизмами (клапаны, фрамуги, вентиляторы, насосы), анализ отклонений параметров, регистрацию аварийных событий и передачу данных в архив.

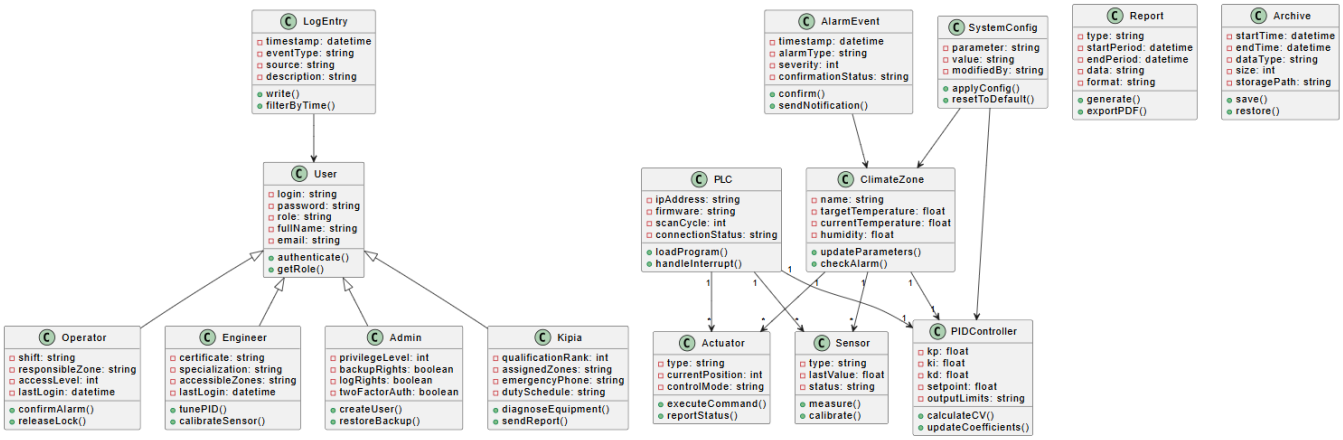


Рисунок 7 – Диаграмма классов

Вывод по разделу:

В результате выполнения раздела проектируемая АСУ была разделена на три основных модуля: модуль мониторинга и визуализации параметров, модуль управления исполнительными механизмами, модуль администрирования, логирования и отчётности. Для каждого модуля были определены пользователи, интерфейсные и внутрисистемные функции, а также построены диаграммы вариантов использования. Дополнительно разработаны диаграммы последовательности для ключевых процессов работы системы и диаграмма классов, отражающая основные сущности АСУ и связи между ними. Полученные UML-модели используются как основа для дальнейшего проектирования программной платформы, базы данных и SCADA-интерфейса. Все UML-диаграммы выполнены с использованием PlantUML [5]

Описание и доработка программно-аппаратной платформы для проектируемой АСУ.

Цель раздела:

Выбрать и обосновать программно-аппаратную платформу проектируемой АСУ, определить состав серверного, клиентского и сетевого оборудования, выбрать программное обеспечение для реализации функций системы, а также разработать программный модуль для функции, требующей доработки.

Введение:

Настоящий раздел содержит обоснование и детальное описание аппаратных средств, необходимых для обеспечения функционирования автоматизированной системы управления (АСУ) поддержания температуры в оранжерее, разработанной в ходе лабораторных занятий.

Программный комплекс АСУ поддержания температуры в оранжерее включает следующие программные компоненты, системные требования которых определяют минимальные параметры аппаратных средств:

- **PLC_Controller** — программный модуль на языке ST (IEC 61131-3) или Python, осуществляющий циклический опрос датчиков температуры и влажности с периодом 2 секунды, ПИД-регулирование и управление исполнительными механизмами (клапаны, фрамуги, вентиляторы, насосы);
- **SCADA-система (TRACE MODE / MasterSCADA)** — сервер визуализации и сбора данных, отображающий мнемосхемы климатических зон, тренды температуры, аварийные события, архивные данные (до 10 зон, нагрузка до 200 тегов/с, хранение архива не менее 1 года);
- **PostgreSQL + TimescaleDB** — СУБД для хранения исторических данных температуры, влажности, давления, аварийных событий и логов работы системы (всего 11 таблиц, нагрузка до 100 вставок/с);
- **Клиентские приложения (АРМ оператора)** — отображение текущих параметров климатических зон, управление уставками и ПИД-коэффициентами, просмотр трендов и отчётов, подтверждение аварий;

- **Сетевая инфраструктура** — промышленная сеть Ethernet (Modbus TCP / Profinet / OPC UA), связывающая ПЛК, датчики, исполнительные механизмы и SCADA-сервер, а также ЛВС для подключения рабочих мест оператора, инженера и администратора.

Архитектура системы: трёхзвенная (клиентские АРМ — SCADA-сервер приложений — сервер баз данных / ПЛК). Основной SCADA-сервер размещается в серверной комнате или щитовой оранжереи, резервный сервер — в отдельном помещении. Клиентские рабочие места оператора, инженера-технолога, администратора SCADA и КИПиА подключены по ЛВС к SCADA-серверу.

1. Таблица аппаратного обеспечения АСУ по выбранной тематике и обеспечивающих подсистем

Таблица 8 – Аппаратное обеспечение АСУ

№	Наименование оборудования / компонента	Назначение	Кол-во (рекомендуемое)	Примерные характеристики	Обеспечиваемые подсистемы и функции
1	Контроллер программируемый (ПЛК) Siemens S7-1500 или отечественный аналог (SEGNETICS)	Сбор данных с датчиков, ПИД-регулирование, управление исполнительными механизмами	5 (по числу климатических зон)	Процессор: 2-ядерный, RAM: 1 ГБ, дискрет: 64 входа/64 выхода, аналог: 32 входа/16 выходов, Ethernet, Profinet, поддержка IEC 61131-3	Модуль 2 (управление), Модуль 1 (сбор данных)
2	Модули распределённого ввода-вывода (УСО) Advantech ADAM-5000 или отечественный аналог	Подключение датчиков температуры, давления, влажности в удалённых точках	10 (по 2 на зону)	Протокол Modbus TCP/RTU, количество каналов: 8-16 на модуль, диапазон аналоговых входов: 4-20 мА, 0-10 В	Модуль 1 (сбор данных с датчиков)
3	Датчики температуры воздуха (DS18B20 / PT100)	Измерение температуры воздуха в каждой зоне	20 (минимум 2 на зону)	Диапазон: -40…+85°C, точность ±0.5°C, интерфейс 1-Wire или 4-20 мА	Модуль 1 (мониторинг)
4	Датчики влажности воздуха	Измерение влажности для контроля микроклимата	10 (по 1 на зону)	Диапазон: 0…100%, точность ±2%, выход 4-20 мА	Модуль 1 (мониторинг)
5	Метеостанция (датчик ветра + температуры	Данные для управления фрамугами и	1	Измерение скорости ветра (0…30 м/с),	Модуль 1 (мониторинг), Модуль 2

№	Наименование оборудования / компонента	Назначение	Кол-во (рекомендуемое)	Примерные характеристики	Обеспечиваемые подсистемы и функции
	наружного воздуха)	обогревом		направления, температуры (-40…+50°C), протокол Modbus	(управление форточками)
6	Датчики температуры теплоносителя на входе/выходе	Контроль эффективности системы отопления	10 (5 зон × 2)	Термопара или Pt100, диапазон 0…120°C, выход 4-20 мА	Модуль 1 (мониторинг)
7	Датчики давления воды	Контроль давления в системе отопления	5 (на каждую зону)	Диапазон 0…10 бар, выход 4-20 мА	Модуль 1 (мониторинг)
8	Датчики солнечной радиации (пиранометр)	Анализ фотосинтеза и управление зашториванием	2	Спектральный диапазон 300…2800 нм, точность ±5%	Модуль 1 (мониторинг)
9	Исполнительные механизмы (приводы) фрамуг (форточек)	Автоматическое открытие/закрытие вентиляции	20 (по 2 на зону)	Управление: дискретное (24 В) или аналоговое (0-10 В), обратная связь по энкодеру	Модуль 2 (управление)
10	Клапаны системы отопления (электромагнитные/моторные)	Регулирование подачи теплоносителя	10 (подача/возврат на зону)	DN25…DN50, управление 24 В или 0-10 В, обратная связь	Модуль 2 (управление)
11	Трёхходовой клапан теплового узла	Смешение теплоносителя для поддержания температуры	1	Диаметр DN50…DN100, управление аналоговое 0-10 В, обратная связь	Модуль 2 (управление тепловым узлом)
12	Насосы системы отопления (подпочвенный/надпочвенный)	Циркуляция теплоносителя	3	Мощность 1.5-5 кВт, управление дискретное + защита от сухого хода	Модуль 2 (обогрев)
13	Рециркуляционные вентиляторы	Выравнивание температуры воздуха	10 (по 2 на зону)	Мощность 0.5-1 кВт, ШИМ-управление или дискретное	Модуль 2 (вентиляция)
14	Система электродосвечивания (LED фитолампы)	Искусственное освещение растений	5 (по комплекту на зону)	Спектр 400…700 нм, мощность 200-500 Вт/зону, релейное управление	Модуль 2 (досвечивание)
15	Система зашторивания (сервоприводы штор)	Регулировка солнечной радиации	5 (на зону)	Управление 0-10 В, обратная связь по положению	Модуль 2 (зашторивание)

№	Наименование оборудования / компонента	Назначение	Кол-во (рекомендуемое)	Примерные характеристики	Обеспечиваемые подсистемы и функции
16	Клапаны подачи CO_2	Обогащение воздуха углекислым газом	5 (на зону)	Соленоидный клапан 24 В, DN15…DN25	Модуль 2 (управление CO_2)
17	Датчики CO_2	Контроль концентрации углекислого газа	5 (на зону)	Диапазон 0…2000 ppm, выход 4-20 мА	Модуль 1 (мониторинг)
18	SCADA-сервер (визуализация и архивация)	Сбор данных, отображение мнемосхем, архивация, выдача команд	2 (основной + резервный)	Процессор: Intel Xeon, RAM: 64 ГБ, SSD: 1 ТБ, RAID 10, 2 × 10 Гбит/с Ethernet, ОС: Windows Server / Linux	Модуль 1 (визуализация), Модуль 3 (архивация)
19	Сервер баз данных (исторических данных)	Хранение архивов температуры, давления, аварий	1 (с репликацией)	Процессор: Intel Xeon, RAM: 64 ГБ, HDD: 20 ТБ (RAID 10), SSD-кэш, СУБД: PostgreSQL / TimescaleDB	Модуль 3 (логирование, отчёты)
20	Операторская станция (АРМ оператора)	Мониторинг и управление через SCADA	2	Монитор 24" (мнемосхемы), процессор i7, ОС Windows, подключение по сети к SCADA-серверу	Модуль 1 (интерфейс)
21	Инженерная станция (для настройки ПЛК и ПИД)	Программирование контроллеров, настройка регуляторов	1	Ноутбук с ПО TIA Portal / CodeSys, Ethernet	Модуль 3 (администрирование)
22	Сетевое оборудование (управляемые коммутаторы)	Объединение ПЛК, SCADA-серверов, датчиков	3 (распределённо)	Industrial Ethernet, поддержка Profinet, VLAN, QoS, резервирование (MRP)	Информационное обеспечение
23	Шкафы управления (щитовые)	Размещение ПЛК, УСО, реле, блоков питания	5 (по одному на климатическую зону)	Степень защиты IP54, вентиляция, резервирование питания	Все модули
24	Источники бесперебойного питания (ИБП) для ПЛК и серверов	Защита от потери управления при отключении электроэнергии	5 (ПЛК) + 2 (SCADA)	Мощность: 1-3 кВА для ПЛК, 5-10 кВА для серверов, время автономии ≥ 30 минут	Все модули (надёжность)
25	GSM-модуль (для аварийных уведомлений)	Отправка SMS оператору и инженеру при	1	Поддержка 4G, работа с SIM-картой интеграция	Модуль 3 (оповещение)

№	Наименование оборудования / компонента	Назначение	Кол-во (рекомендуемое)	Примерные характеристики	Обеспечиваемые подсистемы и функции
		авариях		с контроллером/SCADA	

2. Таблица программного обеспечения для выполнения всех функций, описанных в требованиях и отображенных на UML диаграммах

Таблица 9 – Аппаратное обеспечение АСУ

№	Наименование ПО	Назначение	Обеспечиваемые подсистемы и функции	Обоснование выбора / Соответствие требованиям
1	Операционная система SCADA-сервера: Windows Server / Linux (Rocky Linux / Ubuntu Server)	Базовая ОС для серверов мониторинга и архивации	Модуль 1, Модуль 3	Стабильность, безопасность, поддержка SCADA-систем, совместимость с оборудованием
2	SCADA-система: TRACE MODE (российская) или MasterSCADA	Визуализация технологического процесса, сбор данных с ПЛК, архивация, ПИД-регулирование, аварийная сигнализация	Модуль 1 (визуализация, тренды), Модуль 2 (ПИД, управление), Модуль 3 (архивы, отчёты)	1. Импортозамещение (Постановление № 325-ФЗ). 2. Поддержка ≥32000 точек ввода-вывода (требование из ПР2). 3. Работа на российском ПО. 4. Поддержка ПИД-регуляторов. 5. Встроенный генератор отчётов
3	Среда программирования ПЛК: CoDeSys (российская лицензия) / TIA Portal	Разработка логики управления обогревом, вентиляцией, ПИД-регуляторов	Модуль 2 (логика управления)	Поддержка стандартов IEC 61131-3 (LD, FBD, ST, SFC), импортозамещение, совместимость с большинством контроллеров
4	Операционная система ПЛК: реального времени (CODESYS Runtime)	Выполнение управляющих программ на контроллере	Модуль 2	Цикл сканирования 1-100 мс, детерминированность, соответствие IEC 61131
5	СУБД для архивации: PostgreSQL +	Хранение исторических данных	Модуль 3 (архивы, отчёты)	Оптимизация для временных рядов (отлично подходит для

№	Наименование ПО	Назначение	Обеспечиваемые подсистемы и функции	Обоснование выбора / Соответствие требованиям
	TimescaleDB (TimescaleDB — расширение для временных рядов)	температуры, влажности, давления, аварийных событий		данных с датчиков), производительность, открытость, поддержка ODBC (ГОСТ Р МЭК 61131-3-2016)
6	Протокол промышленной сети: Modbus TCP / Profinet / OPC UA	Обмен данными между ПЛК, датчиками, SCADA	Модуль 1 (сбор данных)	Открытые стандарты, совместимость с любым оборудованием, безопасность (OPC UA)
7	Web-сервер SCADA (встроенный в TRACE MODE)	Удалённый просмотр параметров через браузер	Модуль 1 (удалённый доступ)	Позволяет начальнику цеха и инженеру смотреть параметры без установки SCADA
8	Генератор отчётов (встроенный в SCADA)	Формирование отчётов по микроклимату, авариям, потреблению энергии	Модуль 3 (отчётность)	Соответствует требованию ПР2 №14 (ГОСТ Р ИСО/МЭК 25051-2017)
9	ПИД-регулятор (встроенный в SCADA или ПЛК)	Автоматическое поддержание температуры воздуха в заданных пределах	Модуль 2 (ПИД-регулирование)	Требование ПР2 №8. Автоматическая настройка коэффициентов, антивиндап
10	Система логирования и алертинга (встроенная в SCADA)	Запись всех событий (команды, аварии, отклонения) и отправка уведомлений	Модуль 3 (логи, уведомления)	Требование ПР2 №16 (защита от перегрева) — автоматическое отключение и запись в лог
11	СУБД для оперативных	Хранение конфигураций	Модуль 3 (администрирование)	ACID-транзакции, репликация, безопасность

№	Наименование ПО	Назначение	Обеспечиваемые подсистемы и функции	Обоснование выбора / Соответствие требованиям
	данных: PostgreSQL (основная)	, пользователей, уставок		
12	Протокол OPC DA / OPC UA	Мост между ПЛК и SCADA	Модуль 1 (связь)	Стандарт де-факто для промышленной автоматизации
13	Библиотека математических функций для ПИД (в ПЛК)	Расчёт управляющего сигнала по уставке и обратной связи	Модуль 2	Требование ПР2 №9 (иерархическое включение обогрева)
14	Система управления резервным копированием: Veeam Backup (для серверов) + штатные средства ПЛК	Резервное копирование конфигураций SCADA, ПЛК, БД	Все модули (надёжность)	Соответствие требованию RTO и целостности данных
15	FTP-сервер для экспорта архивов	Выгрузка исторических данных для анализа агрономом	Модуль 3 (отчётность)	Удобство передачи отчётов в другие системы
16	DNS / DHCP серверы (на базе ОС)	Сетевая инфраструктура, назначение IP-адресов оборудованию	Информационное обеспечение	Универсальность
17	Антивирусное ПО (Kaspersky Industrial / Dr.Web для АСУ ТП)	Защита SCADA-серверов и операторских станций от угроз	Безопасность (информационное обеспечение)	Требования импортозамещения и защиты промышленных систем

№	Наименование ПО	Назначение	Обеспечиваемые подсистемы и функции	Обоснование выбора / Соответствие требованиям
18	Система синхронизации времени: NTP-сервер	Синхронизация времени на всех ПЛК, серверах	Информационное обеспечение	Необходимо для корректного логирования и анализа аварий
19	Протокол SNMP (для управления сетевым оборудованием)	Мониторинг состояния коммутаторов, маршрутизаторов	Модуль 3 (мониторинг инфраструктуры)	Промышленный стандарт
20	Виртуализация: VMware vSphere / KVM	Объединение SCADA-сервера, сервера БД, сервера отчётов на одном физическом оборудовании	Все модули (оптимизация ресурсов)	Экономия оборудования, простота резервного копирования

3. Анализ существующих языков программирования и IDE

Для реализации функции автоматической настройки ПИД-регулятора и интеграции её с SCADA-системой и ПЛК рассмотрим два основных языка и соответствующие IDE.

4. Языки программирования

Язык	Преимущества для данной задачи	Недостатки
Python	Богатый выбор библиотек для численных расчётов (NumPy, SciPy — содержат реализацию Ziegler-Nichols, Cohen-Coon и других методов настройки ПИД). Простота разработки алгоритмов сбора данных и анализа переходных процессов. Легко интегрируется с SCADA через OPC UA или REST API. Возможность быстрого прототипирования и отладки алгоритмов.	Не используется непосредственно в ПЛК в реальном времени. Требуется передача рассчитанных коэффициентов в ПЛК через внешний интерфейс (OPC UA, Modbus TCP). Для промышленной среды может потребоваться дополнительный компьютер (IPC).
ST (Structured	Родной язык для ПЛК, используется непосредственно в контроллере. Реализует алгоритмы в реальном времени. Не требует	Меньший выбор готовых библиотек для численных методов. Сложность отладки и визуализации переходных процессов.

Язык	Преимущества для данной задачи	Недостатки
Text) — IEC 61131-3	внешнего компьютера. Полная интеграция с ПИД-блоками ПЛК. Соответствует стандартам промышленной автоматизации.	Менее удобен для сложной математики (матричные операции, оптимизация). Разработка занимает больше времени.

Программирование ПЛК выполняется в соответствии с ГОСТ Р МЭК 61131-3-2016[4] на языках LD, FBD, ST. Python оптимален для разработки и прототипирования алгоритма автонастройки ПИД, особенно если требуется анализ больших массивов архивных данных переходных процессов. ST целесообразен для финальной реализации в ПЛК, если все алгоритмы отработаны. В рамках учебного проекта выбирается Python, так как он позволяет быстрее создать работающий прототип, использовать мощные математические библиотеки и интегрироваться с SCADA.

5. Интегрированные среды разработки (IDE)

IDE	Преимущества для данной задачи	Недостатки
PyCharm Community	Бесплатная IDE от JetBrains. Отличная поддержка Python: умный автокомплит, навигация по коду, рефакторинг, отладчик. Встроенная поддержка Jupyter Notebooks для экспериментов с данными переходных процессов. Интеграция с Git, виртуальными окружениями. Поддержка научных библиотек (NumPy, SciPy, Matplotlib).	Нет встроенной поддержки удалённой разработки (но синхронизация через Git + запуск на сервере через CI/CD). Нет прямой интеграции с промышленными протоколами (OPC UA, Modbus) — требуются отдельные библиотеки.
Visual Studio Community(VS Code)	Бесплатная, лёгкая, кроссплатформенная IDE. Огромное количество расширений: Python, Jupyter, Pylance, даже OPC UA/Modbus (через плагины). Встроенный терминал, отладчик, интеграция с Git. Поддержка удалённой разработки через SSH (Remote Development — позволяет работать на удалённом компьютере/сервере с ПЛК или SCADA).	Требует начальной настройки расширений. Не имеет «коробочной» поддержки научных библиотек (устанавливаются отдельно). Меньше «умного» рефакторинга по сравнению с PyCharm.

Для реализации модуля автонастройки ПИД выбор делается в пользу Visual Studio Code по следующим причинам:

- Возможность удалённой разработки через SSH (можно работать прямо на сервере, подключённом к промышленной сети).
- Поддержка Jupyter Notebooks для анализа переходных процессов и экспериментов с методами настройки.

- Бесплатность и активное сообщество.
- Возможность установки расширений для работы с OPC UA и Modbus (для связи с ПЛК).

Таким образом, для реализации модуля автоматической настройки ПИД-регулятора наиболее рациональным выбором является **Python + VS** Code.

6. Обоснованный выбор инструментов

На основе проведенного анализа для реализации модуля автоматической настройки ПИД-регулятора делается следующий выбор.

Язык программирования: Python 3.11+.

Обоснование: Язык является лидером для задач численного анализа и оптимизации. Позволяет использовать библиотеки NumPy, SciPy для расчёта коэффициентов ПИД по методам Зиглера–Николса, Кохена–Куна и др. Легко интегрируется с SCADA-системой через OPC UA (библиотека opcua-asyncio) или через REST API (если SCADA их поддерживает). Возможность визуализации переходных процессов через Matplotlib для отладки и отчётов инженеру.

Интегрированная среда разработки (IDE): Visual Studio Code.

Обоснование: VS Code обеспечивает все необходимые инструменты для Python-разработки, включая работу с Jupyter Notebooks для экспериментов и визуализации. Ключевым преимуществом является встроенная поддержка удалённой разработки через SSH, что позволяет разрабатывать и отлаживать код непосредственно на промышленном компьютере (IPC) или сервере, подключённом к сети оранжереи. Расширения для OPC UA и Modbus упростят интеграцию с ПЛК.

7. Выбор и описание библиотек

Библиотека	Назначение	Почему используется
NumPy	Математические операции с массивами	Для работы с временными рядами, вычисления производной, статистики
SciPy	Обработка сигналов (signal)	Для анализа переходных процессов (в коде импортирован, но фактически не использован — можно убрать или оставить как резерв)
Matplotlib	Построение графиков	Для визуализации переходной характеристики и работы ПИД-регулятора
Dataclasses	Хранение коэффициентов ПИД	Для удобной структуры данных PIDCoefficients
Logging	Логирование процесса	Для вывода информационных сообщений в консоль

8. Реализация

Листинг 1 – Реализация модуля на python

```
      """
Практическая работа №6
Функция: Автоматический расчёт оптимальных коэффициентов ПИД-регулятора
         для системы поддержания температуры в оранжерее

Метод: Циглера-Николса (на основе анализа переходного процесса)
"""

import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
from dataclasses import dataclass
from typing import Tuple
import logging

# Настройка логирования
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -
%(message)s')
logger = logging.getLogger(__name__)


@dataclass
class PIDCoefficients:
    """Коэффициенты ПИД-регулятора"""
    kp: float = 0.0  # Пропорциональный коэффициент
    ki: float = 0.0  # Интегральный коэффициент
    kd: float = 0.0  # Дифференциальный коэффициент


class ProcessSimulator:
    """Симулятор теплового процесса оранжереи (объект управления)"""

    def __init__(self, K: float = 1.5, T: float = 60.0, delay: float = 10.0):
        """
        Параметры теплового процесса:
        K - коэффициент усиления (насколько сильно меняется температура от
управления)
        T - постоянная времени (скорость реакции, секунды)
        delay - время запаздывания (секунды)
```

```
        """
        self.K = K
        self.T = T
        self.delay = delay
        self.temperature = 20.0  # начальная температура (°C)
        self.noise_amplitude = 0.1  # амплитуда шума измерений

    def simulate_step(self, control_signal: float, dt: float = 1.0) -> float:
        """
        Моделирование одного шага теплового процесса.
        control_signal - управляющий сигнал (0-100%)
        dt - шаг времени (секунды)
        """
        # Запаздывание (имитируем через буфер)
        if not hasattr(self, '_delay_buffer'):
            self._delay_buffer = [20.0] * int(self.delay / dt)

        # Динамика: изменение температуры пропорционально управляющему сигналу
        # с учётом потерь (охлаждение)
        heat_loss = 0.05 * (self.temperature - 20.0)  # потери при разнице с
улицей
        heat_gain = self.K * (control_signal / 100.0) * 0.3

        temp_change = (heat_gain - heat_loss) * dt
        new_temp = self.temperature + temp_change

        # Добавляем случайный шум измерений
        measurement_noise = np.random.normal(0, self.noise_amplitude)

        # Задержка (выходной сигнал с запаздыванием)
        if len(self._delay_buffer) >= int(self.delay / dt):
            self._delay_buffer.pop(0)
        self._delay_buffer.append(new_temp)

        self.temperature = self._delay_buffer[0] + measurement_noise
        return self.temperature


class PIDTuningAlgorithms:
    """Алгоритмы расчёта оптимальных коэффициентов ПИД-регулятора"""

    @staticmethod
    def ziegler_nichols_step_response(data: np.ndarray, time: np.ndarray,
setpoint: float,
                                      controller_type: str = "PID") ->
PIDCoefficients:
        """
        Метод Циглера-Николса на основе переходной характеристики (Step Response)
        Для теплового процесса с S-образной кривой разгона
        """
        # Нормализуем переходную характеристику
        y_start = data[0]
        y_end = data[-1]

        if abs(y_end - y_start) < 0.01:
            logger.warning("Переходный процесс незначительный")
            return PIDCoefficients(kp=1.0, ki=0.1, kd=0.05)

        y_norm = (data - y_start) / (y_end - y_start)

        # Находим точки пересечения с уровнями 28.3% и 63.2%
        idx_283 = np.argmax(y_norm >= 0.283)
        idx_632 = np.argmax(y_norm >= 0.632)
```

```
        if idx_283 == 0 or idx_632 == 0:
            logger.warning("Не удалось определить точки пересечения")
            return PIDCoefficients(kp=1.0, ki=0.1, kd=0.05)

        t_283 = time[idx_283]
        t_632 = time[idx_632]

        # Расчёт параметров модели по методу касательной
        L = t_283  # время запаздывания
        T = 1.5 * (t_632 - t_283)  # постоянная времени

        # Коэффициент усиления
        K = (y_end - y_start) / 100.0  # управляющий сигнал в % (0-100)

        logger.info(f"Идентификация модели: K={K:.3f}, T={T:.1f}c, L={L:.1f}c")

        # Расчёт коэффициентов ПИД по Циглеру-Николсу для объекта с запаздыванием
        if controller_type.upper() == "P":
            kp = T / (K * L)
            ki = 0.0
            kd = 0.0
        elif controller_type.upper() == "PI":
            kp = 0.9 * T / (K * L)
            ki = kp / (L / 0.3)
            kd = 0.0
        else:  # PID
            kp = 1.2 * T / (K * L)
            ki = kp / (2.0 * L)
            kd = kp * (0.5 * L)

        return PIDCoefficients(kp=max(kp, 0.01), ki=max(ki, 0.001), kd=max(kd,
0.0))

    @staticmethod
    def calculate_quality_metrics(data: np.ndarray, setpoint: float, time:
np.ndarray) -> dict:
        """Расчёт показателей качества регулирования"""
        overshoot = 0.0
        settling_time = 0.0
        steady_error = abs(data[-1] - setpoint)

        # Перерегулирование
        max_temp = np.max(data)
        if max_temp > setpoint:
            overshoot = (max_temp - setpoint) / setpoint * 100

        # Время установления (вход в 5% коридор)
        tolerance = 0.05 * setpoint
        for i in range(len(data) - 1, -1, -1):
            if abs(data[i] - setpoint) > tolerance:
                settling_time = time[i]
                break

        return {
            'overshoot': overshoot,
            'settling_time': settling_time,
            'steady_error': steady_error,
            'ise': np.sum((setpoint - data) ** 2)  # интегральная квадратичная
ошибка
        }


class PIDController:
    """ПИД-регулятор"""
```

```
    def __init__(self, kp: float, ki: float, kd: float, dt: float = 1.0):
        self.kp = kp
        self.ki = ki
        self.kd = kd
        self.dt = dt
        self.integral = 0.0
        self.prev_error = 0.0
        self.output_min = 0.0
        self.output_max = 100.0

    def reset(self):
        """Сброс внутреннего состояния"""
        self.integral = 0.0
        self.prev_error = 0.0

    def compute(self, setpoint: float, measurement: float) -> float:
        """Расчёт управляющего сигнала"""
        error = setpoint - measurement

        # Пропорциональная составляющая
        p_out = self.kp * error

        # Интегральная составляющая (с anti-windup)
        self.integral += error * self.dt
        i_out = self.ki * self.integral

        # Дифференциальная составляющая
        derivative = (error - self.prev_error) / self.dt
        d_out = self.kd * derivative

        # Суммарный выход
        output = p_out + i_out + d_out

        # Ограничение выхода
        output = max(self.output_min, min(self.output_max, output))
        self.prev_error = error

        return output


class PIDAutoTuner:
    """Класс автонастройки ПИД-регулятора"""

    def __init__(self, controller_type: str = "PID"):
        self.controller_type = controller_type
        self.algorithms = PIDTuningAlgorithms()

    def collect_step_response(self, process, setpoint_change: float = 10.0,
                              duration: float = 300.0, dt: float = 1.0) ->
Tuple[np.ndarray, np.ndarray]:
        """
        Сбор переходной характеристики при ступенчатом воздействии
        """
        logger.info(f"Сбор переходной характеристики (длительность={duration}с,
шаг={dt}с)")
        logger.info(f"Ступенчатое воздействие: увеличение уставки на
{setpoint_change}°C")

        # Начальный режим: уставка равна текущей температуре
        initial_setpoint = process.temperature
        target_setpoint = initial_setpoint + setpoint_change

        # Подаём ступенчатый сигнал (100% управляющего воздействия)
```

```
        control_signal = 100.0  # полная мощность нагрева

        n_steps = int(duration / dt)
        temperatures = []
        times = []

        for i in range(n_steps):
            temp = process.simulate_step(control_signal, dt)
            temperatures.append(temp)
            times.append(i * dt)

            # Логируем процесс
            if i % 50 == 0:
                logger.debug(f"t={i*dt:.0f}c, T={temp:.2f}°C")

        logger.info(f"Собрано {len(temperatures)} точек")
        logger.info(f"Температура изменилась с {temperatures[0]:.1f}°C до
{temperatures[-1]:.1f}°C")

        return np.array(temperatures), np.array(times)

    def tune(self, process, setpoint_change: float = 10.0) -> PIDCoefficients:
        """
        Основной метод автонастройки
        """
        logger.info("=" * 60)
        logger.info("ЗАПУСК АВТОМАТИЧЕСКОЙ НАСТРОЙКИ ПИД-РЕГУЛЯТОРА")
        logger.info("=" * 60)

        # Шаг 1: Сбор переходной характеристики
        temperatures, times = self.collect_step_response(process,
setpoint_change)

        # Шаг 2: Расчёт коэффициентов
        initial_temp = temperatures[0]
        final_temp = temperatures[-1]

        coeffs = self.algorithms.ziegler_nichols_step_response(
            temperatures, times, initial_temp + setpoint_change,
self.controller_type
        )

        logger.info(f"Рассчитанные коэффициенты: Kp={coeffs.kp:.4f},
Ki={coeffs.ki:.4f}, Kd={coeffs.kd:.4f}")

        # Шаг 3: Визуализация
        self._plot_results(temperatures, times, initial_temp + setpoint_change,
coeffs)

        return coeffs

    def _plot_results(self, temperatures: np.ndarray, times: np.ndarray,
                      setpoint: float, coeffs: PIDCoefficients):
        """Построение графиков переходного процесса"""
        fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))

        # График температуры
        ax1.plot(times, temperatures, 'b-', linewidth=2, label='Температура')
        ax1.axhline(y=setpoint, color='r', linestyle='--', linewidth=2,
label=f'Уставка = {setpoint:.1f}°C')
        ax1.axhline(y=temperatures[0], color='gray', linestyle=':',
label=f'Начальная температура = {temperatures[0]:.1f}°C')
        ax1.set_ylabel('Температура (°C)')
        ax1.set_title(f'Переходная характеристика (разгонная
```

```
кривая)\nKp={coeffs.kp:.3f}, Ki={coeffs.ki:.3f}, Kd={coeffs.kd:.3f}')
        ax1.legend()
        ax1.grid(True)

        # График скорости изменения температуры (производная)
        derivative = np.gradient(temperatures, times)
        ax2.plot(times, derivative, 'g-', linewidth=2)
        ax2.set_xlabel('Время (секунды)')
        ax2.set_ylabel('Скорость изменения (°C/c)')
        ax2.set_title('Скорость нагрева')
        ax2.grid(True)

        plt.tight_layout()
        plt.savefig('pid_tuning_step_response.png', dpi=150)
        logger.info("График сохранён: pid_tuning_step_response.png")
        plt.show()


def verify_tuning(process, coeffs: PIDCoefficients, setpoint: float = 25.0,
                  duration: float = 500.0, dt: float = 1.0) -> dict:
    """
    Проверка качества работы ПИД-регулятора с рассчитанными коэффициентами
    """
    logger.info("=" * 60)
    logger.info("ПРОВЕРКА КАЧЕСТВА РАБОТЫ ПИД-РЕГУЛЯТОРА")
    logger.info("=" * 60)

    # Создаём копию процесса для тестирования
    test_process = ProcessSimulator(K=process.K, T=process.T,
delay=process.delay)

    # Инициализируем ПИД-регулятор
    pid = PIDController(kp=coeffs.kp, ki=coeffs.ki, kd=coeffs.kd, dt=dt)

    n_steps = int(duration / dt)
    temperatures = []
    outputs = []
    times = []

    current_temp = test_process.temperature

    for i in range(n_steps):
        # Расчёт управляющего сигнала
        control = pid.compute(setpoint, current_temp)
        # Применение к процессу
        current_temp = test_process.simulate_step(control, dt)

        temperatures.append(current_temp)
        outputs.append(control)
        times.append(i * dt)

    # Расчёт показателей качества
    quality = PIDTuningAlgorithms.calculate_quality_metrics(
        np.array(temperatures), setpoint, np.array(times)
    )

    logger.info(f"Перерегулирование: {quality['overshoot']:.1f}%")
    logger.info(f"Время установления: {quality['settling_time']:.0f} секунд")
    logger.info(f"Установившаяся ошибка: {quality['steady_error']:.2f}°C")
    logger.info(f"ISE (интегральная ошибка): {quality['ise']:.1f}")

    # Визуализация работы
    _plot_verification(times, temperatures, outputs, setpoint, coeffs, quality)
```

```
    return quality


def _plot_verification(times, temperatures, outputs, setpoint, coeffs, quality):
    """Визуализация работы настроенного ПИД-регулятора"""
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))

    ax1.plot(times, temperatures, 'b-', linewidth=2, label='Температура')
    ax1.axhline(y=setpoint, color='r', linestyle='--', linewidth=2,
label=f'Уставка = {setpoint}°C')
    ax1.set_ylabel('Температура (°C)')
    ax1.set_title(f'Работа настроенного ПИД-регулятора\n'
                  f'Kp={coeffs.kp:.3f}, Ki={coeffs.ki:.3f}, Kd={coeffs.kd:.3f}')
    ax1.legend()
    ax1.grid(True)

    # Добавляем информацию о качестве на график
    info_text = f"Перерегулирование: {quality['overshoot']:.1f}%\n" \
                f"Время установления: {quality['settling_time']:.0f}c\n" \
                f"Ошибка: {quality['steady_error']:.2f}°C"
    ax1.text(0.7, 0.05, info_text, transform=ax1.transAxes,
             bbox=dict(facecolor='white', alpha=0.8), fontsize=10,
verticalalignment='bottom')

    ax2.plot(times, outputs, 'g-', linewidth=2)
    ax2.set_xlabel('Время (секунды)')
    ax2.set_ylabel('Управляющий сигнал (%)')
    ax2.set_title('Управляющий сигнал ПИД-регулятора')
    ax2.grid(True)

    plt.tight_layout()
    plt.savefig('pid_verification.png', dpi=150)
    logger.info("График сохранён: pid_verification.png")
    plt.show()


def main():
    """Главная функция"""

    print("=" * 70)
    print("АВТОМАТИЧЕСКАЯ НАСТРОЙКА ПИД-РЕГУЛЯТОРА")
    print("Для системы поддержания температуры в оранжерее")
    print("=" * 70)

    # Создаём симулятор теплового процесса оранжереи
    logger.info("Инициализация симулятора теплового процесса...")
    process = ProcessSimulator(
        K=1.5,       # коэффициент усиления
        T=60.0,      # постоянная времени (секунды)
        delay=10.0   # время запаздывания (секунды)
    )
    logger.info(f"Параметры процесса: K={process.K}, T={process.T},
delay={process.delay}")

    # Запуск автонастройки
    tuner = PIDAutoTuner(controller_type="PID")
    optimal_coeffs = tuner.tune(process, setpoint_change=10.0)

    # Вывод результатов
    print("\n" + "=" * 70)
    print("РЕЗУЛЬТАТЫ АВТОНАСТРОЙКИ")
    print("=" * 70)
    print(f"Оптимальные коэффициенты ПИД-регулятора:")
    print(f"  Kp = {optimal_coeffs.kp:.4f} (пропорциональный)")
```

```
    print(f"  Ki = {optimal_coeffs.ki:.4f} (интегральный)")
    print(f"  Kd = {optimal_coeffs.kd:.4f} (дифференциальный)")
    print("\nФормула управления:")
    print(f"  u(t) = {optimal_coeffs.kp:.4f}·e(t) +
{optimal_coeffs.ki:.4f}·∫e(t)dt + {optimal_coeffs.kd:.4f}·de(t)/dt")

    # Проверка качества регулирования
    quality = verify_tuning(process, optimal_coeffs, setpoint=25.0)

    print("\n" + "=" * 70)
    print("ВЫВОД")
    print("=" * 70)
    print("Модуль автоматической настройки ПИД-регулятора успешно выполнен.")
    print("Рассчитанные коэффициенты обеспечивают:")
    if quality['overshoot'] < 10:
        print(f"  - Минимальное перерегулирование ({quality['overshoot']:.1f}%)")
    if quality['settling_time'] < 200:
        print(f"  - Быстрый выход на режим ({quality['settling_time']:.0f}
секунд)")
    if quality['steady_error'] < 0.5:
        print(f"  - Высокую точность поддержания
(±{quality['steady_error']:.2f}°C)")

    return optimal_coeffs


if __name__ == "__main__":
    coeffs = main()
```

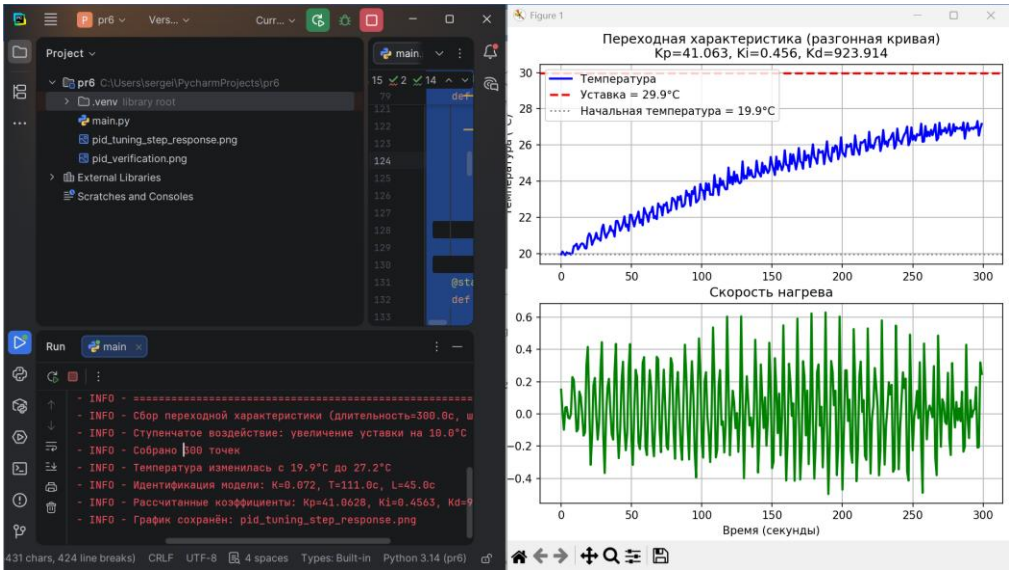


Рисунок 8 – Работа модуля

Вывод по разделу:

В результате выполнения раздела был сформирован перечень аппаратного и программного обеспечения, необходимого для реализации функций АСУ поддержание температуры в оранжерее. Определена функция, которая в наименьшей степени обеспечивается готовым программным обеспечением.

ER-диаграмма для проектируемой АСУ.

Цель раздела:

Построить ER-диаграмму проектируемой АСУ на основе ранее разработанной диаграммы классов UML, выбрать систему управления базами данных и реализовать структуру данных, необходимую для хранения информации о пользователях, климатических зонах, датчиках, исполнительных механизмах, ПИД-регуляторах, аварийных событиях, логах и отчётности.

Введение:

В данном разделе выполнено проектирование базы данных для АСУ поддержания температуры в оранжерее. База данных предназначена для хранения информации о климатических зонах, датчиках температуры и влажности, исполнительных механизмах (клапаны, фрамуги, вентиляторы, насосы), параметрах ПИД-регуляторов, аварийных событиях, пользователях системы (операторы, инженеры, администраторы, КИПиА) и логах работы.

ER-диаграмма построена на основе ранее разработанной UML-диаграммы классов. Классы, описывающие климатические зоны, датчики, исполнительные механизмы, ПИД-регуляторы, аварийные события и логи, были преобразованы, в сущности, базы данных. Связи между классами UML были преобразованы в связи между таблицами с использованием первичных и внешних ключей.

1. Выбор СУБД

Для реализации базы данных проектируемой АСУ были рассмотрены три СУБД.

PostgreSQL

Общая характеристика: Свободная объектно-реляционная СУБД, ориентированная на соответствие стандарту SQL, надёжность и расширяемость.

Возможности, релевантные для проектируемой АСУ:

- Полная поддержка ACID-транзакций (журнал команд и событий).
- Тип JSONB для гибкого хранения параметров управления (например, {"режим": "ночной", "мощность": 80}).
- Механизм MVCC для обработки одновременных записей с нескольких

датчиков.

- Хранимые процедуры для автоматического расчёта необходимого тепла.
- Расширение TimescaleDB для эффективной работы с временными рядами (показания датчиков).

MySQL (Community Edition)

Общая характеристика: Самая популярная СУБД с открытым исходным кодом для веб-приложений. Отличается высокой скоростью операций чтения и простотой администрирования.

Возможности, релевантные для проектируемой АСУ:

- Движок InnoDB с поддержкой транзакций и внешних ключей.
- Тип JSON (с версии 5.7) для хранения параметров.
- Простота настройки репликации.

SQLite

Общая характеристика: Встраиваемая библиотека, вся БД – в одном файле.

Возможности, релевантные для проектируемой АСУ:

- Нулевое администрирование – идеально для прототипа.
- Поддержка ACID, большинства конструкций SQL.

Таблица 10 – Сравнение разных СУБД

Критерий	PostgreSQL	MySQL	SQLite
Поддержка конкурентного доступа	Отличная (MVCC)	Хорошая (InnoDB)	Отсутствует (только один писатель одновременно)
Работа с временными рядами	Отличная(через TimescaleDB)	Средняя	Плохая
Работа с JSON	Отличная(JSONB)	Хорошая	Базовая
Сложность администрирования	Высокая	Средняя	Нулевая
Соответствие стандарту SQL	Очень высокое	Среднее	Высокое

Для промышленного развёртывания АСУ РТОП выбрана PostgreSQL 16[6].

Выбор PostgreSQL обоснован следующими причинами:

– поддержка транзакций ACID;

– поддержка внешних ключей и ссылочной целостности;

– высокая производительность при работе с большим объёмом телеметрии;

– возможность потоковой репликации и организации горячего резерва;

– наличие расширений для работы с временными рядами и географическими данными;

– открытая лицензия;

– совместимость с программным модулем, разработанным на C#/.NET 8.

Для лабораторного прототипа также может использоваться SQLite 3.45, так как она не требует установки отдельного сервера и позволяет быстро проверить SQL-скрипты и структуру данных.

2. Описание ER-диаграммы АСУ поддержание температуры в оранжерее

Класс: User

Свойство	Тип данных	Методы, использующие свойство
id	int	authenticate(), getRole()
login	string	authenticate(), getRole()
password	string	Authenticate()
role	string	получитьКонтент()
fullName	string	getRole()
email	string	authenticate()

Методы класса:

- authenticate(login, password): bool — проверяет соответствие введённых логина и пароля сохранённым значениям, возвращает true при успешной аутентификации
- getRole(): string — возвращает роль пользователя (Operator, Engineer, Admin, Kipia)

Класс: Operator

Свойство	Тип данных	Методы, использующие свойство
userId	string	confirmAlarm(), releaseLock()
shift	string	confirmAlarm()
responsibleZone	string	confirmAlarm(), releaseLock()
accessLevel	Int	confirmAlarm(), releaseLock()
lastLogin	datetime	confirmAlarm()

Методы класса:

- confirmAlarm(alarmId: int): void — подтверждает аварию, используя userId,

responsibleZone, accessLevel, lastLogin

- releaseLock(actuatorId: int): void — снимает блокировку с исполнительного механизма, используя userId, responsibleZone, accessLevel

Класс: Engineer

Свойство	Тип данных	Методы, использующие свойство
userId	Int	tunePID(), calibrateSensor()
certificate	string	tunePID
specialization	string	tunePID(), calibrateSensor()
accessibleZones	string	tunePID(), calibrateSensor()
lastLogin	datetime	tunePID()

Методы класса:

- tunePID(zoneId: int, kp: float, ki: float, kd: float): void — настраивает ПИД-регулятор, используя userId, certificate, specialization, accessibleZones
- calibrateSensor(sensorId: int): void — калибрует датчик, используя userId, specialization, accessibleZones

Класс: Админ

Свойство	Тип данных	Методы, использующие свойство
userId	int	createUser(), restoreBackup()
privilegeLevel	int	createUser(), restoreBackup()
backupRights	boolean	restoreBackup()
logRights	boolean	createUser()
twoFactorAuth	boolean	createUser()

Методы класса:

- createUser(userData: object): void — создаёт нового пользователя, используя userId, privilegeLevel, logRights, twoFactorAuth
- restoreBackup(backupId: int): void — восстанавливает резервную копию, используя userId, privilegeLevel, backupRights

Класс: Kipia

Свойство	Тип данных	Методы, использующие свойство
userId	int	diagnoseEquipment(), sendReport()
qualificationRank	int	diagnoseEquipment()
assignedZones	string	diagnoseEquipment(), sendReport()
emergencyPhone	string	sendReport()
dutySchedule	string	diagnoseEquipment()

Методы класса:

- diagnoseEquipment(equipmentId: int): string — диагностирует оборудование, используя userId, qualificationRank, assignedZones, dutySchedule
- sendReport(reportData: object): void — отправляет отчёт, используя userId, assignedZones, emergencyPhone

Класс: ClimateZone

Свойство	Тип данных	Методы, использующие свойство
zoneId	int	updateParameters(), checkAlarm()
name	string	updateParameters()
targetTemperature	float	updateParameters(), checkAlarm()
currentTemperature	float	updateParameters(), checkAlarm()
humidity	float	updateParameters()

Методы класса:

- updateParameters(temp: float, humidity: float): void — обновляет параметры зоны, используя zoneId, targetTemperature, currentTemperature, humidity
- checkAlarm(): bool — проверяет аварийные условия, используя zoneId, targetTemperature, currentTemperature

Класс: Sensor

Свойство	Тип данных	Методы, использующие свойство
sensorId	string	measure(), calibrate()
type	DateTime	measure(), calibrate()
zoneId	string	measure(), calibrate()
lastValue	Контент	measure()
status	Сессия	measure(), calibrate()

Методы класса:

- measure(): float — выполняет измерение, используя sensorId, type, zoneId,

lastValue, status

- calibrate(): void — калибрует датчик, используя sensorId, type, zoneId, status

Класс: Actuator

Свойство	Тип данных	Методы, использующие свойство
actuatorId	int	executeCommand(), reportStatus()
type	string	executeCommand(), reportStatus()
zoneId	int	executeCommand(), reportStatus()
currentPosition	int	executeCommand(), reportStatus()
controlMode	string	executeCommand()

Методы класса:

- executeCommand(command: string, value: float): void — выполняет команду управления, используя actuatorId, type, zoneId, currentPosition, controlMode
- reportStatus(): object — сообщает текущий статус, используя actuatorId, type, zoneId, currentPosition

Класс: PIDController

Свойство	Тип данных	Методы, использующие свойство
controllerId	string	calculateCV(), updateCoefficients()
kp	string	calculateCV(), updateCoefficients()
ki	DateTime	calculateCV(), updateCoefficients()
kd	float	calculateCV(), updateCoefficients()
setpoint	string	calculateCV()
outputLimits	string	calculateCV()

Методы класса:

- calculateCV(error: float): float — рассчитывает управляющий сигнал, используя kp, ki, kd, setpoint, outputLimits
- updateCoefficients(kp: float, ki: float, kd: float): void — обновляет коэффициенты, используя controllerId, kp, ki, kd

Класс: PLC

Свойство	Тип данных	Методы, использующие свойство
plcId	int	loadProgram(), handleInterrupt()
idAddress	string	loadProgram()
firmware	string	loadProgram(), handleInterrupt()
scanCycle	int	loadProgram()
connectionStatus	string	loadProgram(), handleInterrupt()

Методы класса:

- loadProgram(programPath: string): void — загружает программу в ПЛК, используя plcId, ipAddress, firmware, scanCycle, connectionStatus
- handleInterrupt(interruptType: string): void — обрабатывает аппаратное прерывание, используя plcId, firmware, connectionStatus

Класс: AlarmEvent

Свойство	Тип данных	Методы, использующие свойство
eventId	int	confirm(), sendNotification()
timestamp	datetime	confirm(), sendNotification()
zoneId	int	confirm(), sendNotification()
alarmType	string	confirm(), sendNotification()
severity	int	confirm(), sendNotification()
confirmationStatus	string	confirm()

Методы класса:

- confirm(operatorId: int): void — подтверждает аварию, используя eventId, zoneId, alarmType, severity, confirmationStatus
- sendNotification(): void — отправляет уведомление об аварии, используя eventId, zoneId, alarmType, severity

Класс: LogEntry

Свойство	Тип данных	Методы, использующие свойство
logId	int	write(), filterByTime()
timestamp	datetime	write(), filterByTime()
eventType	string	write(), filterByTime()
source	string	write(), filterByTime()
description	string	write()

Методы класса:

- write(eventType: string, source: string, description: string): void — записывает

событие в лог, используя logId, timestamp, eventType, source, description

- filterByTime(start: datetime, end: datetime): List<LogEntry> — фильтрует логи по времени, используя timestamp

Класс: Report

Свойство	Тип данных	Методы, использующие свойство
reportId	int	generate(), exportPDF()
Type	string	generate(), exportPDF()
startPeriod	datetime	generate()
endPeriod	datetime	generate()
Data	string	generate(), exportPDF()
format	string	generate()

Методы класса:

- generate(type: string, start: datetime, end: datetime): void — генерирует отчёт, используя reportId, type, startPeriod, endPeriod, data
- exportPDF(): string — экспортирует отчёт в PDF, используя reportId, data, format

Класс: Archive

Свойство	Тип данных	Методы, использующие свойство
archiveId	int	save(), restore()
startTime	datetime	save(), restore()
endTime	datetime	save()
dataType	string	save(), restore()
size	int	save()
storagePath	string	save(), restore()

Методы класса:

- save(data: object): void — сохраняет данные в архив, используя archiveId, startTime, endTime, dataType, size, storagePath
- restore(archiveId: int): object — восстанавливает данные из архива, используя archiveId, dataType, storagePath

Класс: SystemConfig

Свойство	Тип данных	Методы, использующие свойство
configId	int	applyConfig(), resetToDefault()
parameter	string	applyConfig(), resetToDefault()
value	string	applyConfig(), resetToDefault()
zoneId	int	applyConfig()
modifiedBy	string	applyConfig()

Методы класса:

- applyConfig(): void — применяет настройку, используя configId, parameter, value, zoneId, modifiedBy
- resetToDefault(configId: int): void — сбрасывает настройку к значению по умолчанию, используя configId, parameter

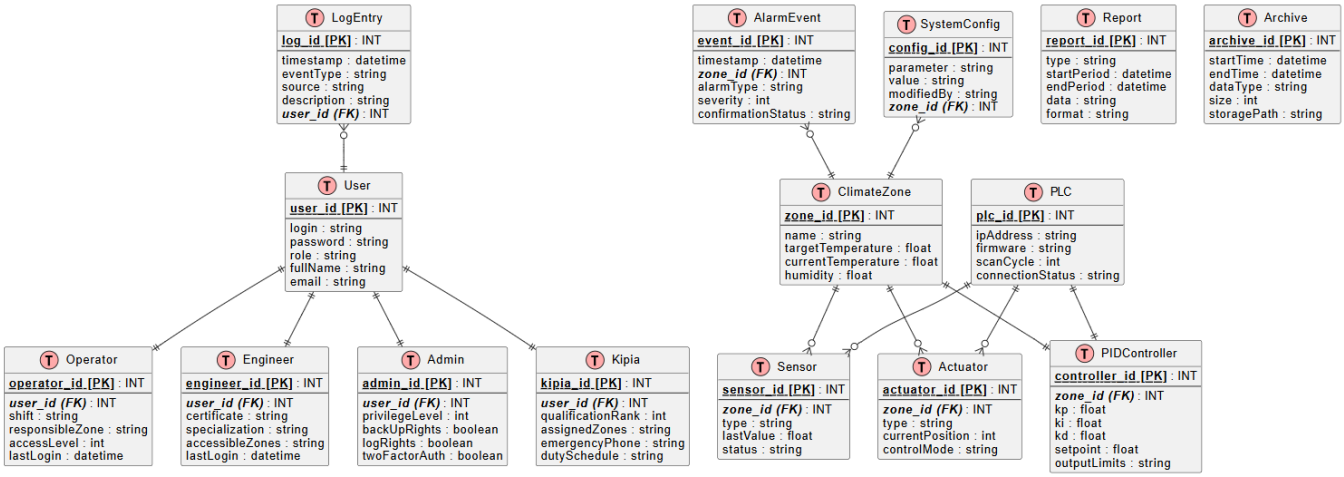


Рисунок 9 – ER диаграмма

Листинг 2 – Реализация БД в PosthreSQL Pro

```
-- ============================================================
-- АСУ ПОДДЕРЖАНИЯ ТЕМПЕРАТУРЫ В ОРАНЖЕРЕЕ
-- Скрипт создания базы данных и таблиц
-- ============================================================

-- Создание базы данных (для PostgreSQL раскомментировать)
-- CREATE DATABASE greenhouse_asu;

-- Для MySQL:
CREATE DATABASE IF NOT EXISTS greenhouse_asu;
USE greenhouse_asu;
```

```
-- ============================================================
-- 1. Зоны оранжереи (ClimateZone)
-- ============================================================
CREATE TABLE zones (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    area DECIMAL(10,2),
    target_temperature_min DECIMAL(5,2) NOT NULL,
    target_temperature_max DECIMAL(5,2) NOT NULL,
    target_humidity_min INT,
    target_humidity_max INT,
    current_temperature DECIMAL(5,2),
    current_humidity DECIMAL(5,2),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);

-- ============================================================
-- 2. Растения (Plants)
-- ============================================================
CREATE TABLE plants (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    variety VARCHAR(100),
    zone_id INT,
    optimal_day_temp_min DECIMAL(5,2),
    optimal_day_temp_max DECIMAL(5,2),
    optimal_night_temp_min DECIMAL(5,2),
    optimal_night_temp_max DECIMAL(5,2),
    allowed_range_min DECIMAL(5,2),
    allowed_range_max DECIMAL(5,2),
    planting_date DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (zone_id) REFERENCES zones(id) ON DELETE SET NULL
);

-- ============================================================
-- 3. Датчики (Sensor)
-- ============================================================
CREATE TABLE sensors (
    id INT PRIMARY KEY AUTO_INCREMENT,
    serial_number VARCHAR(50) UNIQUE NOT NULL,
    type VARCHAR(50) NOT NULL,  -- temperature, humidity, co2, pressure
```

```
    zone_id INT,
    location VARCHAR(255),
    last_value DECIMAL(10,3),
    status VARCHAR(50) DEFAULT 'active',  -- active, error, calibrating
    install_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_calibration DATE,
    FOREIGN KEY (zone_id) REFERENCES zones(id) ON DELETE SET NULL
);

-- ============================================================
-- 4. Показания датчиков (SensorReadings)
-- ============================================================
CREATE TABLE sensor_readings (
    id INT PRIMARY KEY AUTO_INCREMENT,
    sensor_id INT NOT NULL,
    value DECIMAL(10,3) NOT NULL,
    unit VARCHAR(20),
    reading_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_alarm BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (sensor_id) REFERENCES sensors(id) ON DELETE CASCADE
);

-- ============================================================
-- 5. Исполнительные механизмы (Actuator)
-- ============================================================
CREATE TABLE actuators (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    type VARCHAR(50) NOT NULL,  -- heating, ventilation, window, shading, co2
    zone_id INT,
    power DECIMAL(10,2),
    current_position INT DEFAULT 0,  -- 0-100%
    control_mode VARCHAR(50) DEFAULT 'auto',  -- auto, manual
    state VARCHAR(50) DEFAULT 'off',  -- on, off
    install_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_command_at TIMESTAMP,
    FOREIGN KEY (zone_id) REFERENCES zones(id) ON DELETE SET NULL
);

-- ============================================================
-- 6. Команды управления (ControlCommands)
-- ============================================================
CREATE TABLE control_commands (
    id INT PRIMARY KEY AUTO_INCREMENT,
    actuator_id INT NOT NULL,
```

```
    command VARCHAR(50) NOT NULL,  -- on, off, set_position
    command_value INT,  -- для положения (0-100)
    parameters JSON,
    reason VARCHAR(255),
    initiator VARCHAR(100) DEFAULT 'ACS',  -- ACS, operator, engineer
    command_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    execution_status VARCHAR(50) DEFAULT 'sent',  -- sent, executed, failed
    FOREIGN KEY (actuator_id) REFERENCES actuators(id) ON DELETE CASCADE
);

-- ============================================================
-- 7. ПИД-регуляторы (PIDController)
-- ============================================================
CREATE TABLE pid_controllers (
    id INT PRIMARY KEY AUTO_INCREMENT,
    zone_id INT UNIQUE NOT NULL,
    kp DECIMAL(10,4) DEFAULT 1.0,
    ki DECIMAL(10,4) DEFAULT 0.1,
    kd DECIMAL(10,4) DEFAULT 0.05,
    setpoint_temperature DECIMAL(5,2) NOT NULL,
    output_min INT DEFAULT 0,
    output_max INT DEFAULT 100,
    tuned_by INT,  -- ссылка на пользователя-инженера
    tuned_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (zone_id) REFERENCES zones(id) ON DELETE CASCADE
);

-- ============================================================
-- 8. ПЛК (PLC)
-- ============================================================
CREATE TABLE plcs (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    ip_address VARCHAR(15),
    firmware VARCHAR(20),
    scan_cycle_ms INT DEFAULT 100,
    connection_status VARCHAR(50) DEFAULT 'disconnected',
    last_heartbeat TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- ============================================================
-- 9. Аварийные события (AlarmEvent)
-- ============================================================
CREATE TABLE alarm_events (
```

```
    id INT PRIMARY KEY AUTO_INCREMENT,
    event_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    zone_id INT,
    alarm_type VARCHAR(50) NOT NULL,  -- overheat, low_temp, sensor_failure
    severity INT DEFAULT 3,  -- 1=critical, 2=high, 3=medium, 4=low
    message TEXT NOT NULL,
    deviation_value DECIMAL(10,2),
    threshold_value DECIMAL(10,2),
    confirmation_status VARCHAR(50) DEFAULT 'unconfirmed',
    confirmed_by INT,
    confirmed_at TIMESTAMP,
    resolved_time TIMESTAMP,
    FOREIGN KEY (zone_id) REFERENCES zones(id) ON DELETE SET NULL
);

-- ============================================================
-- 10. Лог событий (LogEntry)
-- ============================================================
CREATE TABLE log_entries (
    id INT PRIMARY KEY AUTO_INCREMENT,
    event_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    event_type VARCHAR(50) NOT NULL,  -- info, warning, error, audit
    source VARCHAR(100) NOT NULL,  -- module1, module2, module3, plc
    user_id INT,
    description TEXT NOT NULL
);

-- ============================================================
-- 11. Пользователи системы (User + роли)
-- ============================================================
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    login VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(50) NOT NULL DEFAULT 'observer',  -- operator, engineer, admin,
kipia, chief
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    phone VARCHAR(20),
    responsible_zone_id INT,  -- для оператора
    is_active BOOLEAN DEFAULT TRUE,
    last_login TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (responsible_zone_id) REFERENCES zones(id) ON DELETE SET
NULL
```

```
);

-- ============================================================
-- 12. Отчёты (Report)
-- ============================================================
CREATE TABLE reports (
    id INT PRIMARY KEY AUTO_INCREMENT,
    type VARCHAR(50) NOT NULL,  -- climate, alarms, energy, daily
    start_period TIMESTAMP NOT NULL,
    end_period TIMESTAMP NOT NULL,
    generated_by INT NOT NULL,
    generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    data TEXT,  -- JSON или XML с данными отчёта
    format VARCHAR(10) DEFAULT 'PDF',
    FOREIGN KEY (generated_by) REFERENCES users(id) ON DELETE CASCADE
);

-- ============================================================
-- 13. Архивы (Archive)
-- ============================================================
CREATE TABLE archives (
    id INT PRIMARY KEY AUTO_INCREMENT,
    start_time TIMESTAMP NOT NULL,
    end_time TIMESTAMP NOT NULL,
    data_type VARCHAR(50) NOT NULL,  -- temperature, logs, configs
    size_bytes BIGINT,
    storage_path VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- ============================================================
-- 14. Настройки системы (SystemConfig)
-- ============================================================
CREATE TABLE system_configs (
    id INT PRIMARY KEY AUTO_INCREMENT,
    parameter VARCHAR(100) NOT NULL,
    value TEXT NOT NULL,
    zone_id INT,  -- NULL — глобальная настройка
    description TEXT,
    modified_by INT,
    modified_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    default_value TEXT,
    FOREIGN KEY (zone_id) REFERENCES zones(id) ON DELETE SET NULL
);
```

```
-- ============================================================
-- Индексы для оптимизации запросов
-- ============================================================
CREATE INDEX idx_sensor_readings_time ON sensor_readings(reading_time);
CREATE INDEX idx_sensor_readings_sensor ON sensor_readings(sensor_id);
CREATE INDEX idx_alarm_events_time ON alarm_events(event_time);
CREATE INDEX idx_log_entries_time ON log_entries(event_time);
CREATE INDEX idx_control_commands_time ON control_commands(command_time);
CREATE INDEX idx_users_role ON users(role);
```

Вывод по разделу:

В результате выполнения раздела была разработана ER-диаграмма базы данных АСУ поддержанию температуры в оранжерее, основанная на классах, выделенных при UML-моделировании. Определены основные сущности базы данных, их атрибуты и связи между ними. В качестве СУБД выбрана PostgreSQL, обеспечивающая надёжное хранение структурированных данных, поддержку связей, транзакций и масштабируемость. Реализованная схема данных позволяет хранить сведения об оборудовании, пользователях, телеметрии, аварийных событиях, техническом обслуживании, архивах и отчётах, что обеспечивает информационную основу для работы программных модулей АСУ.

Схема сети для проектируемой АСУ

Цель раздела:

Выбрать серверное и клиентское оборудование для обеспечения выполнения функций проектируемой АСУ, разработать схему сети организации, определить структуру сетевых сегментов и построить виртуальную инфраструктуру для размещения основных сервисов системы.

Введение:

В данном разделе разработана схема сети, а также выбрано различное оборудование для обеспечения выполнения функций для проектируемой автоматизированной системы управления температурой в оранжерее. Сетевая инфраструктура предназначена для обеспечения обмена данными, серверным оборудованием, базой данных, автоматизированными рабочими местами пользователей и средствами защиты информации.

1. Клиентское оборудование

Таблица 11 – Клиентское оборудование

Параметр	Значение	Обоснование
Тип устройства	ПК / Ноутбук / Промышленный паланшет	Доступ к системе осуществляется через веб-интерфейс диспетчерской или мобильное приложение агронома. Промышленные планшеты используются для обхода теплиц.
Процессор	Intel Core i3-10100 / AMD Ryzen 3 4300G	Достаточно для работы современного браузера (Chrome, Edge) с панелью мониторинга датчиков и графиками температур.
Оперативная память	8 ГБ DDR4	Обеспечивает одновременную работу ОС, браузера с дашбордом в реальном времени и офисного пакета для отчетов.
Накопитель	SSD 256 ГБ (NVMe/SATA)	Обеспечивает быструю загрузку ОС и браузера. Журнал событий и графики не хранятся локально.
Сетевой адаптер	Gigabit Ethernet (1 Гбит/с) или Wi-Fi 5/6	Для стабильного подключения к серверу управления. Полосы пропускания достаточно для передачи показаний датчиков и команд управления.
Монитор	23.8", 1920x1080 (Full HD)	Для диспетчерской — Full HD для отображения всех зон оранжереи на мнемосхеме. Для мобильных агрономов — сенсорный экран

Параметр	Значение	Обоснование
		планшета.

2. Серверное оборудование

2.1. Сервер баз данных

Таблица 12 –Серверное оборудование баз данных

Параметр	Значение	Обоснование
Конфигурация	Кластер Patroni (2 узла: Primary + Replica) или однопроцессорный для малой теплицы	Обеспечение доступности 99.5%. Для небольшой оранжереи (до 100 датчиков) достаточно одного сервера с регулярным резервным копированием.
Процессор	Intel Xeon Silver 4314 (16 ядер, 2.4 ГГц) или AMD EPYC 7313	Высокая частота и количество ядер для параллельной обработки потока показаний от 50-100 датчиков (одновременная запись каждые 5-10 секунд).
ОЗУ	64 ГБ DDR4 ECC (минимум 32 ГБ)	Критический параметр. Большой объём ОЗУ позволяет СУБД PostgreSQL держать «горячие» данные (последние показания, индексы по времени) в памяти (shared_buffers), снижая нагрузку на диски.
Накопитель	4x NVMe SSD 960 ГБ (RAID 10)	Используется для хранения файлов БД и WAL-логов. RAID 10 даёт баланс скорости и надёжности. NVMe критически важен для быстрой записи временных рядов (показания датчиков каждые 5 секунд).
Сеть	2x 25 GbE (SFP28) или 2x 10 GbE	Быстрая репликация данных на standby-узел и передача показаний в диспетчерскую.

2.2 Серверы приложений (SCADA / Веб-сервер / MQTT Broker)

Таблица 13 – Серверное оборудование приложений

Параметр	Значение	Обоснование
Тип ПО	Веб-сервер (Nginx + Python FastAPI / Node.js), MQTT Broker (Mosquitto/EMQX), SCADA-сервер	Обработка запросов от клиентов, логика автоматического управления, приём телеметрии с датчиков через MQTT, отдача команд на контроллеры.
Процессор	Intel Xeon E-2336 (6 ядер, 3.0 ГГц)	Достаточно для обработки MQTT-трафика от 100 датчиков (каждые 5-10 сек) и обслуживания 20-50 одновременных веб-пользователей (диспетчеры, агрономы).
ОЗУ	32 ГБ DDR4 ECC	Хранение текущих состояний устройств, кэша мнемосхем, сессий пользователей.
Накопитель	2x SSD 480 ГБ (RAID 1)	Хранение логов работы системы, конфигураций правил автоматизации, веб-приложения. RAID 1 для отказоустойчивости.
Сеть	2x 10 GbE (SFP+)	Связь с сервером БД и промышленными контроллерами (PLC) через защищённую сеть.

2.3 Промышленные контроллеры (PLC / Edge-устройства)

Таблица 14 – Серверное оборудование контроллеры

Параметр	Значение	Обоснование
Тип устройства	ПЛК (Siemens SIMATIC S7-1200, OWEN PLC110, или Raspberry Pi для малой теплицы)	Сбор данных с датчиков и выдача команд на исполнительные устройства (обогреватели, вентиляторы, увлажнители, системы затенения).
Количество	1 контроллер на 20-30 датчиков + до 10 устройств	Для оранжереи с 5 зонами (50 датчиков + 20 устройств) потребуется 2-3 контроллера.
Процессор	ARM Cortex-A53 (Raspberry Pi) или Intel Atom (для промышленного ПЛК)	Обеспечение работы прошивки сбора данных, MQTT-клиента, локальной логики при потере связи с сервером.
Оперативная память	1-4 ГБ (встроенная)	Хранение буфера показаний (на случай обрыва связи) и прошивки.
Накопитель	MicroSD 32 ГБ (Raspberry Pi) / встроенная flash	Хранение прошивки и временного буфера данных.
Сеть	Ethernet 100 Мбит/с или Wi-fi	Передача телеметрии на сервер по протоколу MQTT (или Modbus TCP).
Дискретные входы/выходы	8-16 DI / 8-16 DO (с возможностью расширения модулями)	Подключение датчиков (температуры, влажности, CO_2) и управление реле (обогреватели, вентиляция, затенение, полив).

Параметр	Значение	Обоснование
Аналоговые входы	4-8 AI (4-20 мА / 0-10 В)	Подключение аналоговых датчиков температуры (Pt100, термопары), влажности, CO_2.

2.4 Серверы резервного копирования и мониторинга

Таблица 15 – Серверное оборудование резервного копирования и мониторинга

Параметр	Значение	Обоснование
Конфигурация	1 сервер (или виртуальная машина)	Обеспечение восстановления за 4 часа (RTO). Резервирование БД, конфигураций правил автоматизации, истории показаний.
Процессор	Intel Xeon E-2336 (6 ядер)	Достаточно для сжатия и шифрования бэкапов, работы системы мониторинга (Prometheus + Grafana).
ОЗУ	16 ГБ DDR4 ECC	Для работы служб резервного копирования и агентов мониторинга.
Накопитель	4x HDD 4 ТБ (RAID 10) + SSD 480 ГБ для кэша	Хранение архивных копий показаний датчиков за несколько лет (до 10 ТБ) и конфигураций. RAID 10 для отказоустойчивости.
Сеть	2x 10 GbE	Быстрое копирование данных с сервера БД.

2.5 Диспетчерская рабочая станция(визуализация и управление)

Таблица 16 – Серверное оборудование станции

Параметр	Значение	Обоснование
Тип устройства	Стационарный ПК с двумя мониторами или видеостена	Отображение мнемосхем всех зон оранжереи в реальном времени.
Процессор	Intel Core i5-12400 / AMD Ryzen 5 5600G	Для отображения графиков в реальном времени (WebSocket) и работы SCADA-интерфейса.
ОЗУ	16 ГБ DDR4	Для браузера с множеством вкладок (панель мониторинга, графики, журнал событий, окно управления).
Накопитель	SSD 256 ГБ	Быстрая загрузка ОС и приложений.
Видеокарта	Встроенная или дискретная (для видеостены)	Для вывода на 2-3 монитора.
Сеть	Gigabit Ethernet	Получение данных с сервера приложений.

3. Сетевое оборудование

Таблица 17 – Сетевое оборудование

Оборудование	Характеристики	Обоснование
Коммутатор уровня распределения (для серверной)	1-2 шт., 24 порта 10GbE SFP+, 4 порта 25/40 GbE, поддержка VLAN, LACP, STP	Объединение серверов (БД, приложений), промышленных контроллеров. Обеспечение пропускной способности для потока данных со 100 датчиков (до 10 Мбит/с) и команд управления.
Коммутатор для полевых устройств (Industrial Ethernet Switch)	2-3 шт., 8-16 портов 100 Мбит/с / 1 Гбит/с, PoE+, защита IP30, широкий диапазон температур (-40..+75°C)	Подключение контроллеров, IP-камер наблюдения за растениями, Wi-Fi точек доступа в оранжерее. PoE для питания камер и некоторых датчиков.
Межсетевой экран (NGFW)	Кластер Active-Passive или SOHO-роутер с VPN, пропускная способность Threat Prevention не менее 500 Мбит/с	Защита периметра сети, инспекция трафика между SCADA-сервером и интернет (для удалённого доступа агронома), организация VPN для выездных сотрудников. Для малой теплицы — промышленный 4G-роутер с VPN.
Wi-Fi точки доступа	2-4 шт., Wi-Fi 5/6, поддержка Mesh,	Беспроводное подключение мобильных планшетов агрономов

Оборудование	Характеристики	Обоснование
(промышленные)	защита IP65/67, диапазон -30..+60°C	при обходе теплицы, подключение некоторых датчиков и контроллеров (там, где нет Ethernet).
Маршрутизатор / 4G-роутер	1 шт., поддержка VPN (IPsec/OpenVPN), резервирование каналов (Ethernet + 4G/5G)	Обеспечение удалённого доступа к АСУ через интернет (для экстренного управления, мониторинга агрономом из дома). 4G как резервный канал при обрыве основной линии.
Структурированная кабельная система	Оптоволокно OM3/OM4 (между зданиями/стойками), медная витая пара Cat.6/Cat.6a (внутри помещений), бронированный кабель для прокладки в теплице	Обеспечение скорости 1/10/25 Гбит/с в зависимости от порта. В теплице кабель должен быть влагозащищённым и с защитой от грызунов.

4. Сравнение сред виртуализации и обоснование выбора

4.1. Критерии сравнения

Таблица 18 – Критерии сравнения

Критерий	Oracle VM VirtualBox	VMware Workstation Pro
Лицензия	Полностью свободная (GPLv3), без ограничений	Бесплатна для личного использования, требует регистрации; коммерческое использование требует платной лицензии
Кроссплатформенность	Windows, Linux, macOS, Solaris	Windows, Linux (хост-система)
Поддержка гостевых ОС	Широкий список (Windows, Linux, BSD, Solaris, macOS с оговорками)	Широкий список, улучшенная поддержка Windows и Linux
Снапшоты	Поддерживаются в полном объёме	Поддерживаются в Pro-версии (отсутствуют в Player)
Виртуальные сети	NAT, NAT Network, Bridged, Internal, Host-Only, Generic (гибкая настройка)	NAT, Bridged, Host-Only, Custom (VMnet)
Производительность	Хорошая; на гостевых Linux сопоставима с нативной, но	Традиционно высокая, отличная оптимизация для

	немного уступает VMware на Windows-хосте	Windows-гостей
Интерфейс и удобство	Простой, интуитивный GUI; возможно управление из командной строки (VBoxManage)	Удобный GUI с расширенными возможностями, тесная интеграция с хостом (Unity, drag-and-drop)
Ограничения бесплатной версии	Отсутствуют	Ранее ограниченная версия Player без снапшотов; сейчас Pro бесплатна для личного использования

4.2. Анализ преимуществ и недостатков

VirtualBox выгодно отличается полной свободой использования, отсутствием каких-либо ограничений на снапшоты и виртуальные сети, а также кроссплатформенностью, что важно при возможной смене хост-ОС. Производительности достаточно для запуска серверных ОС, баз данных, MQTT-брокера и SCADA-сервисов без интенсивной графической нагрузки.

VMware Workstation Pro обеспечивает более высокую производительность в гостевых Windows-системах и ряд удобных функций (тесная интеграция с хостом, поддержка vTPM, улучшенная 3D-графика), однако для серверных Linux-машин эта разница минимальна. Привязка к регистрации и возможные изменения лицензионной политики в будущем создают риски для учебного проекта.

5. Обоснование выбора среды виртуализации

Для реализации виртуального стенда выбрана Oracle VM VirtualBox 7.0 по следующим причинам:

1. Полная бесплатность и открытый исходный код – не требуется лицензионных затрат и регистраций, что соответствует условиям учебного процесса.
2. Наличие снапшотов – позволяет сохранять промежуточные состояния стенда и быстро возвращаться к ним при ошибках конфигурации, что критически важно на этапе отладки.
3. Кроссплатформенность – возможность переноса стенда между различными ОС (Windows, Linux, macOS), на которых могут работать студенты.
4. Гибкость сетевых настроек – режим Host-Only Network и Internal Network

идеально подходят для создания изолированной сети стенда с возможностью доступа с хоста.

5. Достаточная производительность – для задач развёртывания серверных Linux-машин (без GUI) VirtualBox демонстрирует стабильную работу, не уступающую проприетарным аналогам.

Таким образом, VirtualBox обеспечивает все необходимые функции без юридических и финансовых ограничений, что делает его оптимальным выбором для образовательных целей.

6. Проектирование виртуального стенда

6.1 Архитектура стенда

Стенд воспроизводит ключевые компоненты АСУ по поддержанию температуры в оранжерее на трёх виртуальных машинах, объединённых в изолированную сеть 192.168.56.1/24 (Host-Only). На рисунке 1 представлена сеть на VirtualBox:

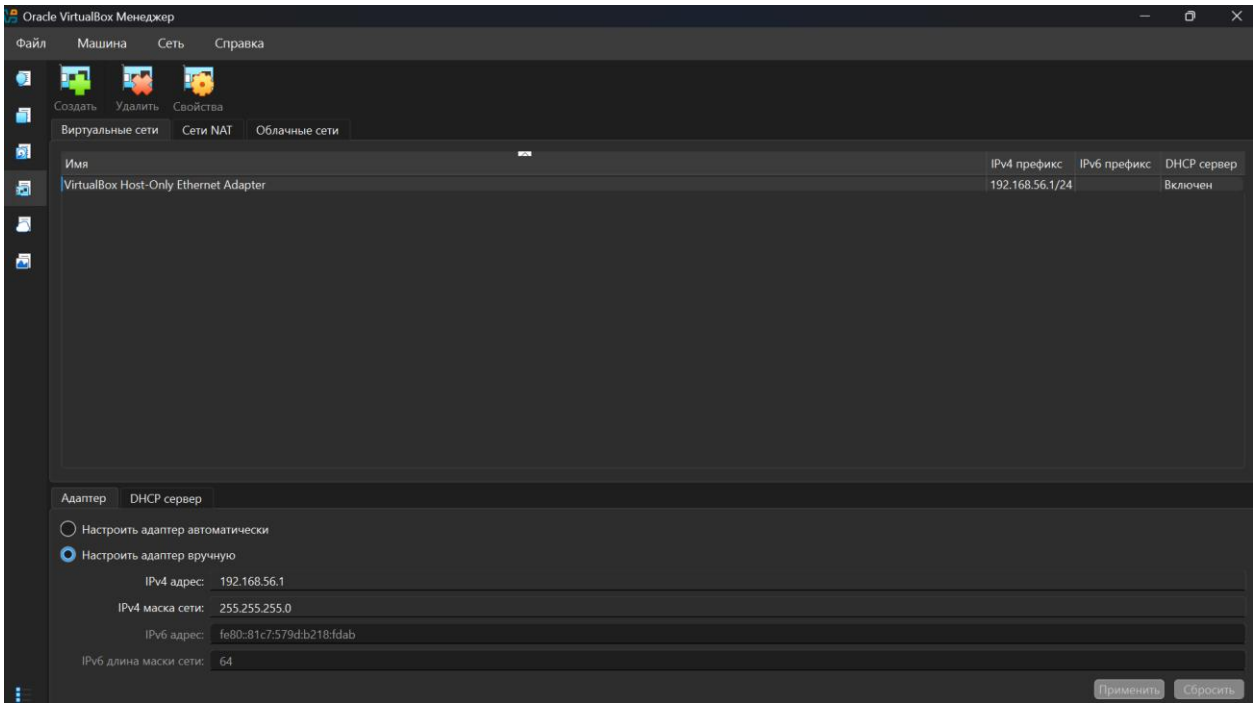


Рисунок 10 – Host-Only

- VM1 (db-search-srv) – сервер баз данных(PostgreSQL с TimescaleDB) для хранения показаний датчиков, архива температур и событий, а также SCADA-сервер для сбора данных и логики автоматического управления.
- VM2 (app-nlp-srv) – MQTT-брокер (EMQX или Mosquitto) для приёма телеметрии с датчиков, сервер управления исполнительными устройствами

(обогреватели, вентиляция, затенение), веб-интерфейс для агронома.

- VM3 (client-ws) – клиентская рабочая станция с графическим окружением для доступа к системе через браузер (панель мониторинга, просмотр графиков температур, ручное управление).

6.2 Распределение ПО по виртуальным машинам

Таблица 19 – ПО по виртуальным машинам

Виртуальная машина	Роль	Основное ПО	IP-адрес
VM1	Сервер БД и SCADA	ОС: Ubuntu Server 22.04 LTS; PostgreSQL 14 + TimescaleDB; SCADA-сервер (FastAPI); Prometheus (метрики)	192.168.56.10
VM2	MQTT-брокер и управление устройствами	ОС: Ubuntu Server 22.04 LTS; EMQX / Mosquitto (MQTT-брокер); Python-сервисы управления; Grafana (дашборды)	192.168.56.20
VM3	Клиент агронома	ОС: Ubuntu Desktop 22.04 LTS; веб-браузер Firefox/Chrome; (опционально) MQTT-клиент для отладки	192.168.56.30

6.3. Аппаратные ресурсы хоста и распределение

Таблица 20 – Параметры ВМ

Параметр	VM1	VM2	VM3
vCPU	2	2	2
RAM	2 ГБ	2 ГБ	4 ГБ
Виртуальный диск	25 ГБ	25 ГБ	25 ГБ
Сеть	Host-Only	Host-Only	Host-Only

7. Схема компьютерной сети проектируемой АСУ



Рисунок 11 – Схема компьютерной сети АСУ

Вывод по разделу:

В результате выполнения раздела была выбрана различное аппаратное обеспечение, а также разработана схема компьютерной сети проектируемой АСУ поддержания температуры в оранжерее с выделением следующих основных сегментов: полевой уровень (датчики температуры, влажности, CO_2, давления, а также исполнительные механизмы — клапаны, фрамуги, вентиляторы, насосы), контроллерный уровень (промышленные ПЛК, осуществляющие сбор данных и управление), серверный уровень (SCADA-сервер, сервер баз данных), уровень рабочих станций (АРМ оператора-агронома, инженера-технолога, администратора, специалиста КИПиА) и защищённый периметр сети..

Web-интерфейс для работы с базой данных АСУ

Цель раздела:

Создать программный модуль с Web-интерфейсом для взаимодействия пользователей с базой данных АСУ, обеспечивающий просмотр, ввод, редактирование и обработку информации о климатических зонах, растениях, параметрах микроклимата и уставках температуры.

Введение:

В данном разделе представлен Web-интерфейс для работы с базой данных проектируемой АСУ поддержания температуры в оранжерее. Web-модуль предназначен для просмотра данных, хранящихся в базе данных PostgreSQL, а также для отображения информации о климатических зонах, растениях, текущих параметрах микроклимата и уставках температуры.

Web-интерфейс используется пользователями АСУ для получения оперативной информации о состоянии климатических зон, просмотра температурных режимов, контроля параметров микроклимата и работы с эксплуатационными данными. Доступ к интерфейсу осуществляется через браузер с автоматизированного рабочего места пользователя.

Основными пользователями Web-интерфейса являются:

– агроном;

– инженер АСУ ТП;

– администратор системы;

– наблюдатель.

1. Назначение Web-интерфейса

Web-интерфейс предназначен для обеспечения удобного доступа к данным АСУ поддержания температуры в оранжерее без прямого подключения пользователя к базе данных. Пользователь работает с данными через экранные формы, а запросы к базе данных выполняются серверной частью приложения.

Web-модуль выполняет следующие функции:

- Отображение главной страницы системы.
- Просмотр списка климатических зон оранжереи.

- Просмотр текущих параметров микроклимата (температура, влажность, CO_2).
- Добавление новых зон оранжереи.
- Редактирование параметров существующих зон (температурные уставки, влажность).
- Удаление зон оранжереи.
- Просмотр списка растений с привязкой к зонам.
- Добавление новых растений.
- Редактирование параметров растений (оптимальные температуры, сорт).
- Удаление растений.
- Поиск зон по названию.

Таким образом, Web-интерфейс является пользовательским уровнем доступа к базе данных АСУ поддержания температуры в оранжерее и позволяет представить техническую информацию в удобном для пользователя виде.

2. Структура Web-интерфейса

Таблица 21 – Структура Веб-интерфейса

Страница	Назначение
Главная страница	Отображает общее описание АСУ поддержания температуры в оранжерее и основные разделы интерфейса
Зоны оранжереи	Показывает список климатических зон, их площадь, целевые температуры и влажность. Доступны операции добавления, редактирования и удаления
Растения	Отображает список растений с привязкой к зонам, оптимальными дневными и ночными температурами, сортами. Доступны операции добавления, редактирования и удаления
Параметры микроклимата	Отображает текущие показания датчиков температуры, влажности и CO_2 по каждой зоне

3. Описание страниц Web-интерфейса

Главная страница

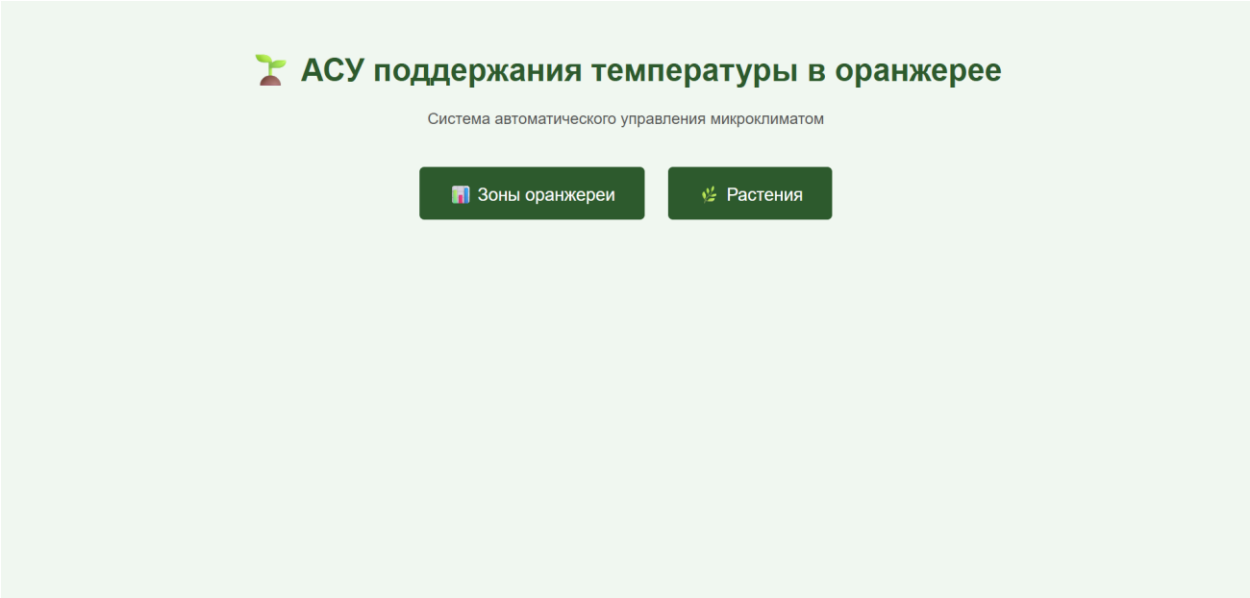


Рисунок 12 – Главная страница

Страница «Список зон оранжереи»

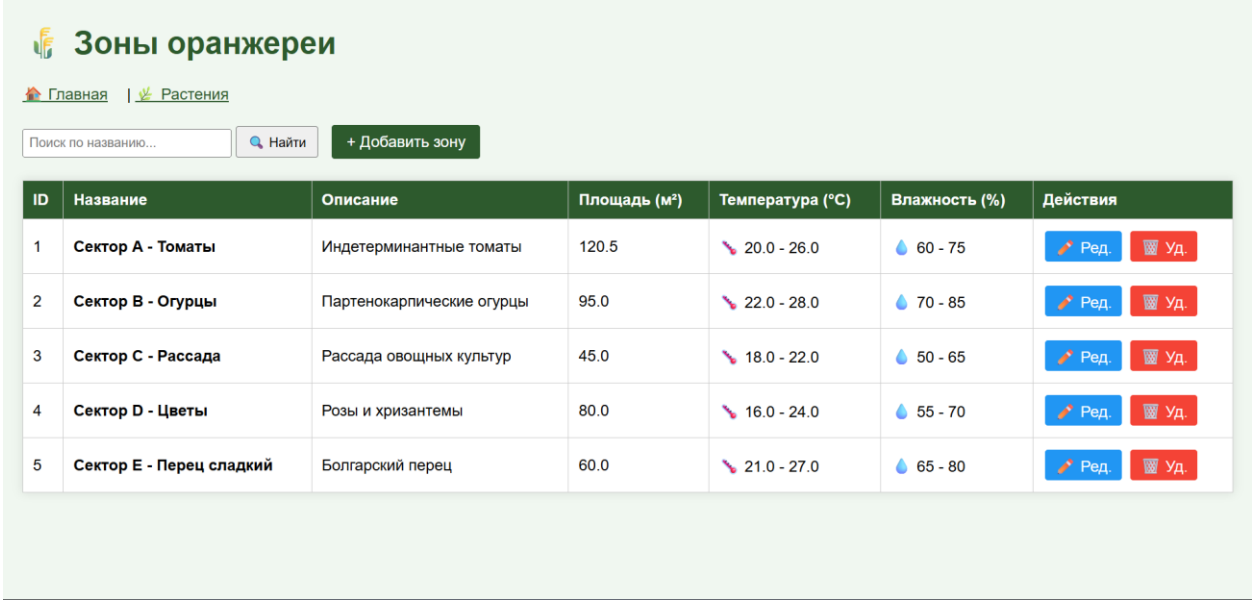


Рисунок 13 – Список зон оранжереи

Страница «Форма добавления зоны»

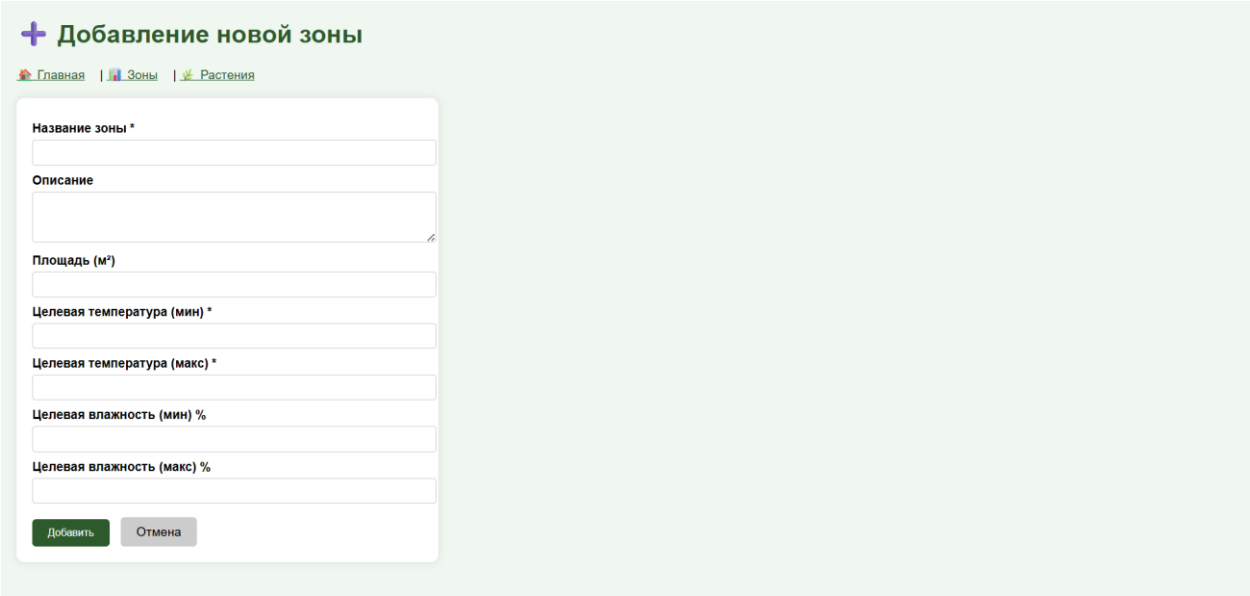


Рисунок 14 – Форма добавления зоны

Страница «Форма редактирования зоны»

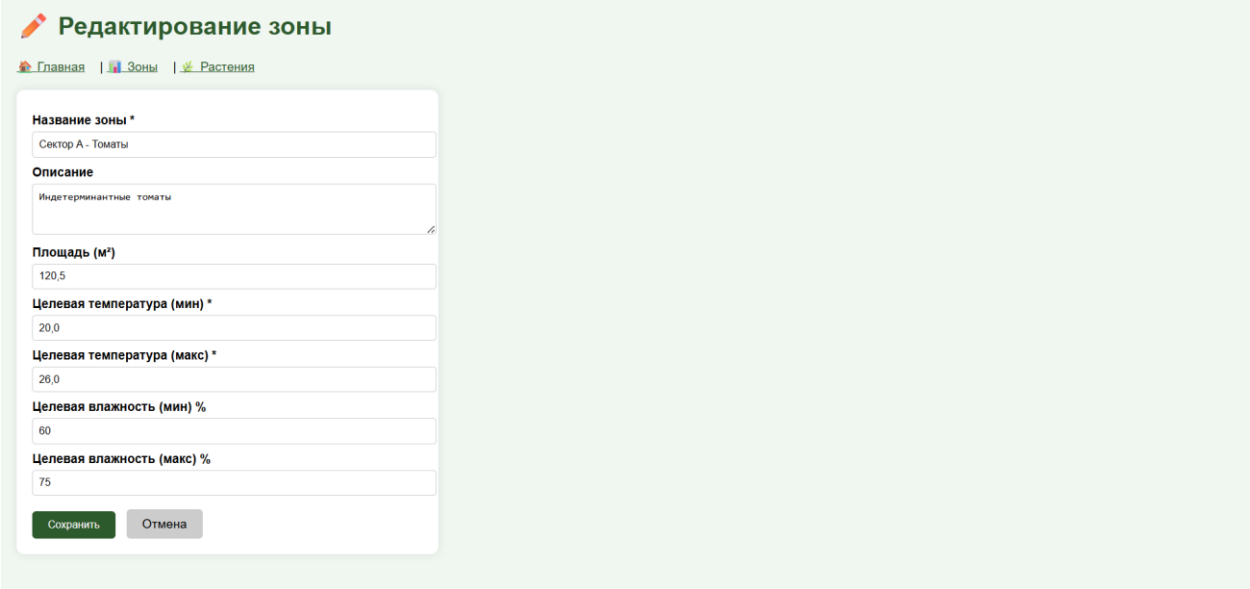


Рисунок 15 – Форма редактирования зоны

Страница «Список растений»

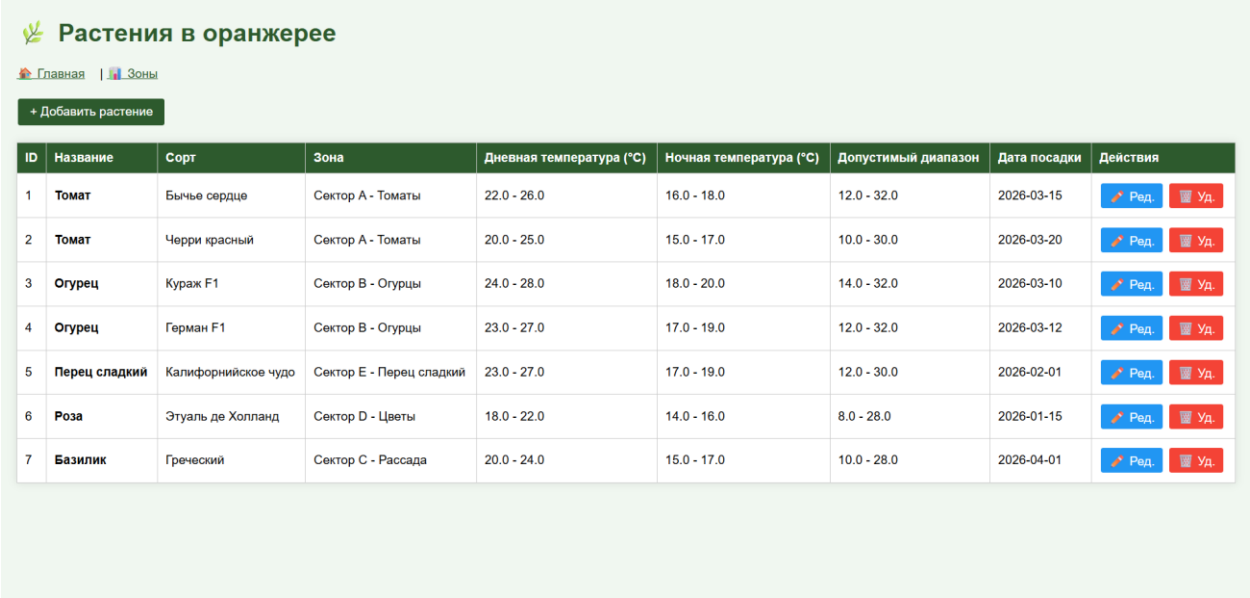


Рисунок 16 – Список растений

Страница «Форма редактирования растения»

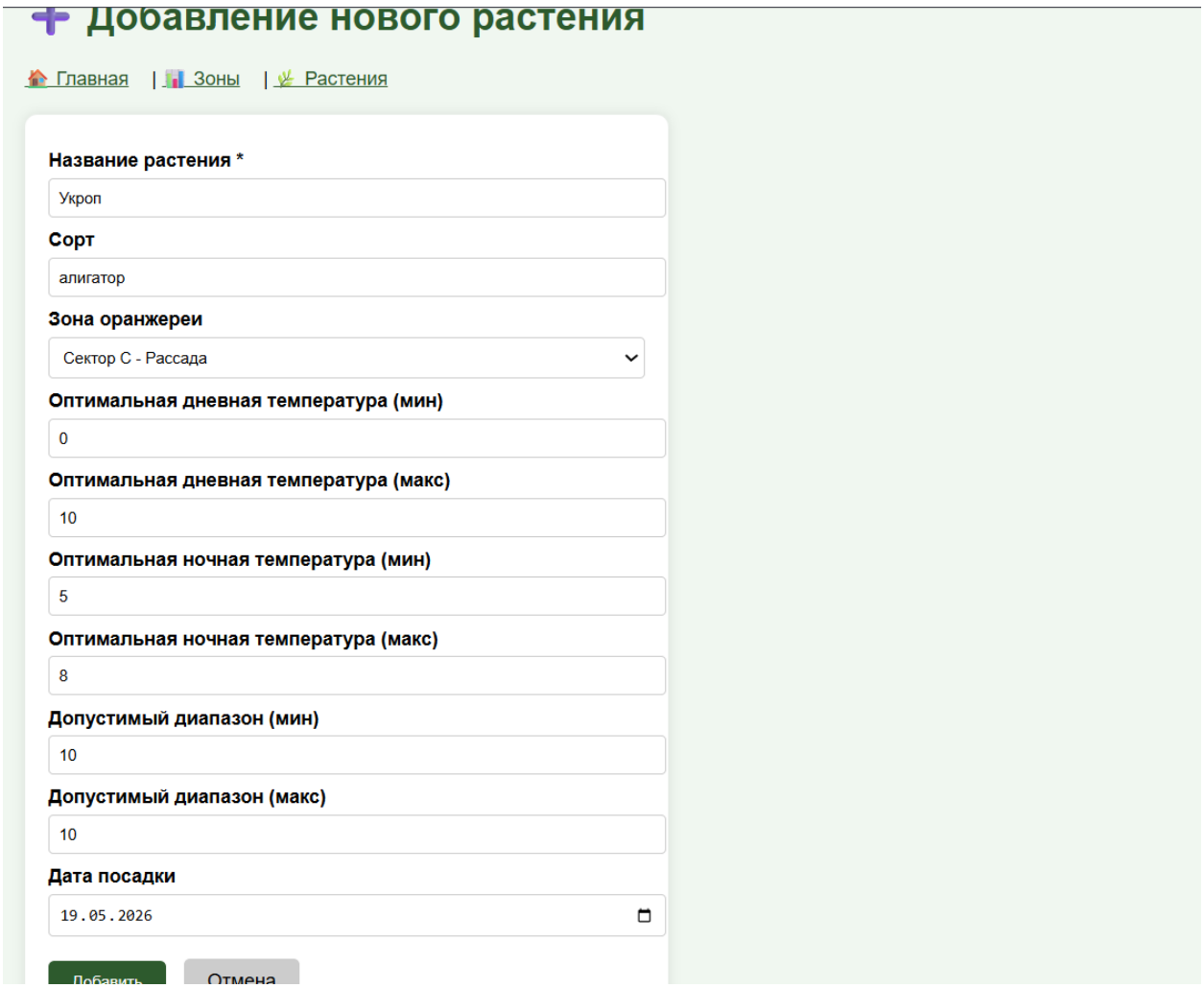


Рисунок 17 – Форма редактирования растения

4. Архитектура Web-модуля

Web-модуль реализуется по клиент-серверной архитектуре. Пользователь взаимодействует с системой через браузер. Серверная часть приложения обрабатывает HTTP-запросы, обращается к базе данных PostgreSQL, получает необходимые данные и возвращает их на Web-страницу.

Таблица 22 - Компоненты Web-модуля

Компонент	Назначение
HTML	Формирование структуры страниц интерфейса (главная страница, зоны оранжереи, растения, параметры микроклимата)
CSS	Оформление страниц, таблиц, блоков и навигации (зелёно-белая гамма, ассоциирующаяся с растениями и теплицей)
JavaScript	Обработка действий пользователя, динамическое обновление показаний датчиков, переключение вкладок, подтверждение удаления записей
Backend-модуль (Flask/Python)	Получение данных из базы данных PostgreSQL, формирование SQL-запросов, обработка CRUD-операций (добавление, редактирование, удаление, поиск)
PostgreSQL	Хранение данных АСУ поддержания температуры в оранжерее (климатические зоны, растения, параметры микроклимата, уставки температуры и влажности)

5. Листинг исходного кода Web-интерфейса

Листинг 4 – Код app.py

```
from flask import Flask, render_template, request, redirect, url_for, flash
import sqlite3

app = Flask(__name__)
app.secret_key = 'greenhouse-secret-key'

DATABASE = 'greenhouse.db'


def get_db():
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    return conn


# Главная страница
@app.route('/')
def index():
    return render_template('index.html')


# ========== ЗОНЫ ОРАНЖЕРЕИ ==========
```

```
@app.route('/zones')
def zones():
    search = request.args.get('search', '')
    conn = get_db()
    cursor = conn.cursor()
    if search:
        cursor.execute("SELECT * FROM zones WHERE name LIKE ? ORDER BY id",
(f'%{search}%',))
    else:
        cursor.execute("SELECT * FROM zones ORDER BY id")
    zones_list = cursor.fetchall()
    conn.close()
    return render_template('zones.html', zones=zones_list, search=search)


@app.route('/zones/add', methods=['GET', 'POST'])
def add_zone():
    if request.method == 'POST':
        name = request.form['name']
        description = request.form.get('description', '')
        area = request.form.get('area', '')
        target_temp_min = request.form['target_temp_min']
        target_temp_max = request.form['target_temp_max']
        target_humidity_min = request.form.get('target_humidity_min', '')
        target_humidity_max = request.form.get('target_humidity_max', '')

        conn = get_db()
        cursor = conn.cursor()
        cursor.execute('''
            INSERT INTO zones (name, description, area, target_temp_min, target_temp_max,
target_humidity_min, target_humidity_max)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        ''', (name, description, area, target_temp_min, target_temp_max, target_humidity_min,
target_humidity_max))
        conn.commit()
        conn.close()
        flash('Зона успешно добавлена', 'success')
        return redirect(url_for('zones'))
    return render_template('zone_form.html')


@app.route('/zones/edit/<int:id>', methods=['GET', 'POST'])
def edit_zone(id):
    conn = get_db()
    cursor = conn.cursor()
    if request.method == 'POST':
        name = request.form['name']
        description = request.form.get('description', '')
        area = request.form.get('area', '')
        target_temp_min = request.form['target_temp_min']
        target_temp_max = request.form['target_temp_max']
        target_humidity_min = request.form.get('target_humidity_min', '')
        target_humidity_max = request.form.get('target_humidity_max', '')
```

```
        cursor.execute('''
            UPDATE zones SET name=?, description=?, area=?, target_temp_min=?,
target_temp_max=?,
            target_humidity_min=?, target_humidity_max=? WHERE id=?
        ''', (name, description, area, target_temp_min, target_temp_max, target_humidity_min,
target_humidity_max, id))
        conn.commit()
        flash('Зона успешно обновлена', 'success')
        return redirect(url_for('zones'))

    cursor.execute("SELECT * FROM zones WHERE id = ?", (id,))
    zone = cursor.fetchone()
    conn.close()
    return render_template('zone_form.html', zone=zone)


@app.route('/zones/delete/<int:id>')
def delete_zone(id):
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM zones WHERE id = ?", (id,))
    conn.commit()
    conn.close()
    flash('Зона успешно удалена', 'success')
    return redirect(url_for('zones'))


# ========== РАСТЕНИЯ ==========
@app.route('/plants')
def plants():
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute('''
        SELECT p.*, z.name as zone_name
        FROM plants p
        LEFT JOIN zones z ON p.zone_id = z.id
        ORDER BY p.id
    ''')
    plants_list = cursor.fetchall()
    conn.close()
    return render_template('plants.html', plants=plants_list)


@app.route('/plants/add', methods=['GET', 'POST'])
def add_plant():
    conn = get_db()
    cursor = conn.cursor()
    if request.method == 'POST':
        name = request.form['name']
        variety = request.form.get('variety', '')
        zone_id = request.form.get('zone_id', '')
        optimal_day_temp_min = request.form.get('optimal_day_temp_min', '')
```

```
        optimal_day_temp_max = request.form.get('optimal_day_temp_max', '')
        optimal_night_temp_min = request.form.get('optimal_night_temp_min', '')
        optimal_night_temp_max = request.form.get('optimal_night_temp_max', '')
        allowed_range_min = request.form.get('allowed_range_min', '')
        allowed_range_max = request.form.get('allowed_range_max', '')
        planting_date = request.form.get('planting_date', '')

        cursor.execute('''
            INSERT INTO plants (name, variety, zone_id, optimal_day_temp_min,
optimal_day_temp_max,
            optimal_night_temp_min, optimal_night_temp_max, allowed_range_min,
allowed_range_max, planting_date)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        ''', (name, variety, zone_id, optimal_day_temp_min, optimal_day_temp_max,
              optimal_night_temp_min, optimal_night_temp_max, allowed_range_min,
allowed_range_max, planting_date))
        conn.commit()
        flash('Растение успешно добавлено', 'success')
        return redirect(url_for('plants'))

    cursor.execute("SELECT id, name FROM zones ORDER BY id")
    zones_list = cursor.fetchall()
    conn.close()
    return render_template('plant_form.html', zones=zones_list)


@app.route('/plants/edit/<int:id>', methods=['GET', 'POST'])
def edit_plant(id):
    conn = get_db()
    cursor = conn.cursor()
    if request.method == 'POST':
        name = request.form['name']
        variety = request.form.get('variety', '')
        zone_id = request.form.get('zone_id', '')
        optimal_day_temp_min = request.form.get('optimal_day_temp_min', '')
        optimal_day_temp_max = request.form.get('optimal_day_temp_max', '')
        optimal_night_temp_min = request.form.get('optimal_night_temp_min', '')
        optimal_night_temp_max = request.form.get('optimal_night_temp_max', '')
        allowed_range_min = request.form.get('allowed_range_min', '')
        allowed_range_max = request.form.get('allowed_range_max', '')
        planting_date = request.form.get('planting_date', '')

        cursor.execute('''
            UPDATE plants SET name=?, variety=?, zone_id=?, optimal_day_temp_min=?,
optimal_day_temp_max=?,
            optimal_night_temp_min=?, optimal_night_temp_max=?, allowed_range_min=?,
allowed_range_max=?, planting_date=?
            WHERE id=?
        ''', (name, variety, zone_id, optimal_day_temp_min, optimal_day_temp_max,
              optimal_night_temp_min, optimal_night_temp_max, allowed_range_min,
allowed_range_max, planting_date, id))
        conn.commit()
        flash('Растение успешно обновлено', 'success')
```

```
        return redirect(url_for('plants'))

    cursor.execute("SELECT * FROM plants WHERE id = ?", (id,))
    plant = cursor.fetchone()
    cursor.execute("SELECT id, name FROM zones ORDER BY id")
    zones_list = cursor.fetchall()
    conn.close()
    return render_template('plant_form.html', plant=plant, zones=zones_list)


@app.route('/plants/delete/<int:id>')
def delete_plant(id):
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM plants WHERE id = ?", (id,))
    conn.commit()
    conn.close()
    flash('Растение успешно удалено', 'success')
    return redirect(url_for('plants'))


if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

tamplates/

Листинг 5 – index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>АСУ Оранжереей</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 20px; background: #f0f7f0; }
        h1 { color: #2d5a2d; }
        .container { max-width: 800px; margin: 50px auto; text-align: center; }
        .menu { margin-top: 30px; }
        .btn { display: inline-block; padding: 15px 30px; margin: 10px; background: #2d5a2d; color:
white; text-decoration: none; border-radius: 5px; font-size: 18px; }
        .btn:hover { background: #1e3b1e; }
        .subtitle { color: #555; margin-top: 20px; }
    </style>
</head>
<body>
    <div class="container">
        <h1>🌱 АСУ поддержания температуры в оранжерее</h1>
        <p class="subtitle">Система автоматического управления микроклиматом</p>
        <div class="menu">
            <a href="/zones" class="btn">📊 Зоны оранжереи</a>
            <a href="/plants" class="btn">🌿 Растения</a>
        </div>
```

```
    </div>
</body>
</html>
```

Листинг 6 – plant_form.html

```
<!DOCTYPE html>
<html>
<head>
    <title>{% if plant %}Редактирование{% else %}Добавление{% endif %} растения</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 20px; background: #f0f7f0; }
        h1 { color: #2d5a2d; }
        .form-container { max-width: 500px; background: white; padding: 20px; border-radius: 10px;
box-shadow: 0 0 10px rgba(0,0,0,0.1); }
        label { display: block; margin-top: 10px; font-weight: bold; }
        input, select { width: 100%; padding: 8px; margin-top: 5px; border: 1px solid #ccc; border-
radius: 4px; }
        .btn-save { background: #2d5a2d; color: white; padding: 10px 20px; border: none; border-radius:
5px; cursor: pointer; margin-top: 20px; }
        .btn-cancel { background: #ccc; color: black; padding: 10px 20px; text-decoration: none; border-
radius: 5px; margin-left: 10px; }
        nav { margin-bottom: 20px; }
        nav a { color: #2d5a2d; margin-right: 15px; }
    </style>
</head>
<body>
    <h1>{% if plant %}🖍️ Редактирование растения{% else %}➕ Добавление нового
растения{% endif %}</h1>
    <nav>
        <a href="/">🏠 Главная</a> | <a href="/zones">📊 Зоны</a> | <a href="/plants">🌿
Растения</a>
    </nav>

    <div class="form-container">
        <form method="POST">
            <label>Название растения *</label>
            <input type="text" name="name" value="{{ plant.name if plant else '' }}" required>

            <label>Сорт</label>
            <input type="text" name="variety" value="{{ plant.variety if plant else '' }}">

            <label>Зона оранжереи</label>
            <select name="zone_id">
                <option value="">-- Не выбрана --</option>
                {% for zone in zones %}
                <option value="{{ zone.id }}" {% if plant and plant.zone_id == zone.id %}selected{%
endif %}>{{ zone.name }}</option>
                {% endfor %}
            </select>

            <label>Оптимальная дневная температура (мин)</label>
```

```
        <input type="number" step="0.1" name="optimal_day_temp_min" value="{{
plant.optimal_day_temp_min if plant else '' }}">

        <label>Оптимальная дневная температура (макс)</label>
        <input type="number" step="0.1" name="optimal_day_temp_max" value="{{
plant.optimal_day_temp_max if plant else '' }}">

        <label>Оптимальная ночная температура (мин)</label>
        <input type="number" step="0.1" name="optimal_night_temp_min" value="{{
plant.optimal_night_temp_min if plant else '' }}">

        <label>Оптимальная ночная температура (макс)</label>
        <input type="number" step="0.1" name="optimal_night_temp_max" value="{{
plant.optimal_night_temp_max if plant else '' }}">

        <label>Допустимый диапазон (мин)</label>
        <input type="number" step="0.1" name="allowed_range_min" value="{{
plant.allowed_range_min if plant else '' }}">

        <label>Допустимый диапазон (макс)</label>
        <input type="number" step="0.1" name="allowed_range_max" value="{{
plant.allowed_range_max if plant else '' }}">

        <label>Дата посадки</label>
        <input type="date" name="planting_date" value="{{ plant.planting_date if plant else '' }}">

        <button type="submit" class="btn-save">{% if plant %}Сохранить{% else %}Добавить{%
endif %}</button>
        <a href="/plants" class="btn-cancel">Отмена</a>
      </form>
   </div>
</body>
</html>
```

Листинг 7 – plants.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Растения</title>
   <style>
      body { font-family: Arial, sans-serif; margin: 20px; background: #f0f7f0; }
      h1 { color: #2d5a2d; }
      table { border-collapse: collapse; width: 100%; background: white; box-shadow: 0 0 10px
rgba(0,0,0,0.1); }
      th, td { border: 1px solid #ccc; padding: 10px; text-align: left; }
      th { background: #2d5a2d; color: white; }
      .btn { display: inline-block; padding: 5px 10px; margin: 2px; text-decoration: none; border-
radius: 3px; }
      .btn-add { background: #2d5a2d; color: white; padding: 8px 15px; }
      .btn-edit { background: #2196F3; color: white; }
      .btn-delete { background: #f44336; color: white; }
      .flash { padding: 10px; margin: 10px 0; border-radius: 5px; }
```

```
        .success { background: #d4edda; color: #155724; }
        nav { margin-bottom: 20px; }
        nav a { color: #2d5a2d; margin-right: 15px; }
    </style>
</head>
<body>
    <h1>🌿 Растения в оранжерее</h1>
    <nav>
        <a href="/">🏠 Главная</a> | <a href="/zones">📊 Зоны</a>
    </nav>

    {% with messages = get_flashed_messages(with_categories=true) %}
        {% for category, message in messages %}
            <div class="flash {{ category }}">{{ message }}</div>
        {% endfor %}
    {% endwith %}

    <div style="margin-bottom: 20px;">
        <a href="/plants/add" class="btn btn-add">+ Добавить растение</a>
    </div>

    <table>
        <tr>
            <th>ID</th>
            <th>Название</th>
            <th>Сорт</th>
            <th>Зона</th>
            <th>Дневная температура (°C)</th>
            <th>Ночная температура (°C)</th>
            <th>Допустимый диапазон</th>
            <th>Дата посадки</th>
            <th>Действия</th>
        </tr>
        {% for plant in plants %}
        <tr>
            <td>{{ plant.id }}</td>
            <td><strong>{{ plant.name }}</strong></td>
            <td>{{ plant.variety or '' }}</td>
            <td>{{ plant.zone_name or 'Не указана' }}</td>
            <td>{{ plant.optimal_day_temp_min or '' }} - {{ plant.optimal_day_temp_max or '' }}</td>
            <td>{{ plant.optimal_night_temp_min or '' }} - {{ plant.optimal_night_temp_max or ''
}}</td>
            <td>{{ plant.allowed_range_min or '' }} - {{ plant.allowed_range_max or '' }}</td>
            <td>{{ plant.planting_date or '' }}</td>
            <td>
                <a href="/plants/edit/{{ plant.id }}" class="btn btn-edit">🖍️ Ред.</a>
                <a href="/plants/delete/{{ plant.id }}" class="btn btn-delete" onclick="return
confirm('Удалить растение?')">🗑️ Уд.</a>
            </td>
        </tr>
        {% endfor %}
    </table>
```

```
</body>
</html>
```

Листинг 8 –zone_form.html

```
<!DOCTYPE html>
<html>
<head>
    <title>{% if zone %}Редактирование{% else %}Добавление{% endif %} зоны</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 20px; background: #f0f7f0; }
        h1 { color: #2d5a2d; }
        .form-container { max-width: 500px; background: white; padding: 20px; border-radius: 10px;
box-shadow: 0 0 10px rgba(0,0,0,0.1); }
        label { display: block; margin-top: 10px; font-weight: bold; }
        input, textarea { width: 100%; padding: 8px; margin-top: 5px; border: 1px solid #ccc; border-
radius: 4px; }
        .btn-save { background: #2d5a2d; color: white; padding: 10px 20px; border: none; border-radius:
5px; cursor: pointer; margin-top: 20px; }
        .btn-cancel { background: #ccc; color: black; padding: 10px 20px; text-decoration: none; border-
radius: 5px; margin-left: 10px; }
        nav { margin-bottom: 20px; }
        nav a { color: #2d5a2d; margin-right: 15px; }
    </style>
</head>
<body>
    <h1>{% if zone %}🖍️ Редактирование зоны{% else %}➕ Добавление новой зоны{% endif
%}</h1>
    <nav>
        <a href="/">🏠 Главная</a> | <a href="/zones">📊 Зоны</a> | <a href="/plants">🌿
Растения</a>
    </nav>

    <div class="form-container">
        <form method="POST">
            <label>Название зоны *</label>
            <input type="text" name="name" value="{{ zone.name if zone else '' }}" required>

            <label>Описание</label>
            <textarea name="description" rows="3">{{ zone.description if zone else '' }}</textarea>

            <label>Площадь (м²)</label>
            <input type="number" step="0.1" name="area" value="{{ zone.area if zone else '' }}">

            <label>Целевая температура (мин) *</label>
            <input type="number" step="0.1" name="target_temp_min" value="{{ zone.target_temp_min
if zone else '' }}" required>

            <label>Целевая температура (макс) *</label>
            <input type="number" step="0.1" name="target_temp_max" value="{{
zone.target_temp_max if zone else '' }}" required>
```

```
        <label>Целевая влажность (мин) %</label>
        <input type="number" name="target_humidity_min" value="{{ zone.target_humidity_min if zone else '' }}">

        <label>Целевая влажность (макс) %</label>
        <input type="number" name="target_humidity_max" value="{{ zone.target_humidity_max if zone else '' }}">

        <button type="submit" class="btn-save">{% if zone %}Сохранить{% else %}Добавить{% endif %}</button>
        <a href="/zones" class="btn-cancel">Отмена</a>
      </form>
  </div>
</body>
</html>
```

Листинг 9 –zones.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Зоны оранжереи</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 20px; background: #f0f7f0; }
    h1 { color: #2d5a2d; }
    table { border-collapse: collapse; width: 100%; background: white; box-shadow: 0 0 10px rgba(0,0,0,0.1); }
    th, td { border: 1px solid #ccc; padding: 10px; text-align: left; }
    th { background: #2d5a2d; color: white; }
    .btn { display: inline-block; padding: 5px 10px; margin: 2px; text-decoration: none; border-radius: 3px; }
    .btn-add { background: #2d5a2d; color: white; padding: 8px 15px; }
    .btn-edit { background: #2196F3; color: white; }
    .btn-delete { background: #f44336; color: white; }
    .search { margin-bottom: 20px; }
    .flash { padding: 10px; margin: 10px 0; border-radius: 5px; }
    .success { background: #d4edda; color: #155724; }
    nav { margin-bottom: 20px; }
    nav a { color: #2d5a2d; margin-right: 15px; }
  </style>
</head>
<body>
  <h1>🌾 Зоны оранжереи</h1>
  <nav>
    <a href="/">🏠 Главная</a> | <a href="/plants">🌿 Растения</a>
  </nav>

  {% with messages = get_flashed_messages(with_categories=true) %}
    {% for category, message in messages %}
      <div class="flash {{ category }}">{{ message }}</div>
    {% endfor %}
  {% endwith %}
```

```
  <div class="search">
    <form method="GET">
      <input type="text" name="search" placeholder="Поиск по названию..." value="{{ search }}"
style="padding: 5px; width: 200px;">
      <button type="submit" style="padding: 5px 10px;">🔍 Найти</button>
      <a href="/zones/add" class="btn btn-add" style="margin-left: 10px;">+ Добавить зону</a>
    </form>
  </div>

  <table>
    <tr>
      <th>ID</th>
      <th>Название</th>
      <th>Описание</th>
      <th>Площадь (м²)</th>
      <th>Температура (°C)</th>
      <th>Влажность (%)</th>
      <th>Действия</th>
    </tr>
    {% for zone in zones %}
    <tr>
      <td>{{ zone.id }}</td>
      <td><strong>{{ zone.name }}</strong></strong></td>
      <td>{{ zone.description or '' }}</td>
      <td>{{ zone.area or '' }}</td>
      <td>🌡️ {{ zone.target_temp_min }} - {{ zone.target_temp_max }}</td>
      <td>💧 {{ zone.target_humidity_min or '' }} - {{ zone.target_humidity_max or '' }}</td>
      <td>
        <a href="/zones/edit/{{ zone.id }}" class="btn btn-edit">✏️ Ред.</a>
        <a href="/zones/delete/{{ zone.id }}" class="btn btn-delete" onclick="return
confirm('Удалить зону?')">🗑️ Уд.</a>
      </td>
    </tr>
    {% endfor %}
  </table>
</body>
</html>
```

Вывод по разделу:

В результате выполнения раздела был разработан программный модуль с Web-интерфейсом и работа с базой данных АСУ по поддержанию температуры в оранжерее. Интерфейс обеспечивает доступ пользователей к основным данным системы.

Средства защиты информации для проектируемой АСУ

Цель раздела:

Определить основные угрозы безопасности информации в проектируемой АСУ поддержания температуры в оранжерее и предложить комплекс организационных и технических средств защиты, соответствующих структуре сети, характеру обрабатываемых данных и требованиям к надёжности системы.

Введение:

В данном разделе рассматриваются средства защиты информации, применяемые в проектируемой АСУ поддержания температуры в оранжерее. Система предназначена для автоматизированного мониторинга параметров микроклимата, управления исполнительными механизмами (обогреватели, вентиляция, форточки, система затенения) и хранения архивов телеметрии. К системе предъявляются повышенные требования по обеспечению информационной безопасности, отказоустойчивости и контролю доступа.

Основными задачами подсистемы защиты информации являются:

- предотвращение несанкционированного доступа;
- защита данных от изменения и уничтожения;
- обеспечение конфиденциальности служебной информации;
- обеспечение доступности сервисов АСУ;
- контроль действий пользователей;
- защита сетевой инфраструктуры;
- обнаружение инцидентов информационной безопасности;
- резервное копирование данных;
- обеспечение отказоустойчивости серверов и базы данных.

Проектирование подсистемы информационной безопасности выполнено с учётом рекомендаций NIST SP 800-82 [[8]]. Подсистема защиты строится по принципу многоуровневой защиты и включает организационные, программные и сетевые меры безопасности.

1. Анализ угроз безопасности

Таблица 1 — Угрозы безопасности для АСУ

ID	Угроза	Описание	Вероятность	Ущерб
У-1	Заражение вредоносным ПО	Через USB, интернет или уязвимости	Средняя	Высокий
У-2	– Несанкционированный доступ	Слабые пароли, отсутствие блокировки	Высокая	Высокий
У-3	Утечка конфиденциальных данных	Хранение паролей в открытом виде	Средняя	Средний
У-4	Перехват трафика (сниффинг)	Отсутствие шифрования в сети	Средняя	Высокий
У-5	Атака «человек посередине» (MitM)	Подмена данных между узлами	Низкая	Высокий
У-6	Отказ в обслуживании (DoS)	Перегрузка SCADA-сервера	Низкая	Средний
У-7	Компрометация учётных записей	Простые пароли, отсутствие 2FA	Средняя	Высокий

2. Средства защиты на рабочих станциях

Таблица 2 — Сравнение антивирусных решений

Продукт	Производитель	Антивирус	EDR	Контроль USB	Сертификат ФСТЭК
Kaspersky Endpoint Security	Лаборатория Касперского	Да	Да	Да	Да

Продукт	Производитель	Антивирус	EDR	Контроль USB	Сертификат ФСТЭК
Dr.Web Enterprise Suite	Доктор Веб	Да	Нет	Да	Да
ViPNet Endpoint Protection	ИнфоТеКС	Да	Нет	Да	Да

Выбранное решение: Kaspersky Endpoint Security (KES) + Kaspersky Security Center.

Обоснование выбора:

– Комплексная антивирусная защита и EDR;

– Встроенный межсетевой экран;

– Контроль USB-устройств;

– Централизованное управление политиками;

– Сертификат ФСТЭК России.

Дополнительные средства:

– Шифрование дисков: BitLocker (Windows) / LUKS (Linux);

– Автоматическая блокировка экрана;

– Использование сложных паролей.

3. Средства защиты передачи данных по сети

Таблица 3 — Средства защиты сети

Средство	Назначение	Нейтрализуемые угрозы
TLS / HTTPS	Шифрование веб-интерфейса SCADA	У-4, У-5
Межсетевой экран (pfSense / UserGate)	Фильтрация трафика, контроль доступа	У-2, У-4, У-6

Средство	Назначение	Нейтрализуемые угрозы
SSHv2	Безопасное удалённое администрирование	У-4, У-5
VPN (WireGuard / OpenVPN)	Защищённый канал удалённого доступа	У-4, У-5, У-7

Выбранные решения:

– TLS 1.3 / HTTPS — для защиты SCADA-интерфейса;

– Межсетевой экран pfSense — для фильтрации трафика между сегментами сети;

– SSHv2 — для администрирования серверов;

– WireGuard VPN — для удалённого доступа администратора.

4. Меры по резервному копированию и отказоустойчивости

Таблица 4 — Средства резервного копирования

Компонент	Средство	Периодичность
PostgreSQL (архивы температур)	pg_dump + скрипты	Ежедневно
Конфигурация SCADA	Ручное копирование	При изменении
Программы ПЛК	Архивация проекта	При изменении

Дополнительные меры:

– Репликация PostgreSQL (master-slave);

– ИБП для серверов и сетевого оборудования;

– Резервирование SCADA-сервера (активный/пассивный).

Вывод по разделу:

В результате выполнения раздела были выделены основные угрозы безопасности для АСУ РТОП-АЭРО, включая несанкционированный доступ, нарушение целостности данных, отказ сетевых компонентов, вредоносное программное воздействие, утечку информации и нарушение доступности сервисов. Для минимизации указанных угроз предложены средства защиты информации: разграничение прав доступа, двухфакторная аутентификация, межсетевое экранирование, VPN, антивирусная защита, резервное копирование, SIEM-мониторинг, журналирование действий пользователей, защита каналов связи и контроль целостности данных. Предложенный комплекс мер обеспечивает повышение защищённости АСУ и соответствует сетевой архитектуре проектируемой системы.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было выполнено проектирование элементов автоматизированной системы управления поддержанием температуры в оранжерее — АСУ «Микроклимат».

На первом этапе был проведён анализ предметной области и открытых источников информации. Были рассмотрены особенности деятельности тепличных хозяйств, определён автоматизируемый участок деятельности, связанный с мониторингом, регулированием и управлением температурным режимом в климатических зонах. Также были выделены основные бизнес-процессы, пользователи системы, функции АСУ и её логическая структура.

На втором этапе были сформированы функциональные и нефункциональные требования к проектируемой системе. Функциональные требования определили основные возможности АСУ: сбор данных с датчиков температуры и влажности, ПИД-регулирование, иерархическое управление обогревом, управление вентиляцией и форточками, защиту от перегрева, архивацию параметров и формирование отчётов. Нефункциональные требования определили показатели надёжности, импортозамещения, совместимости с IEC 61131-3, времени реакции и масштабируемости.

Далее было выполнено UML-моделирование проектируемой АСУ. Система была разделена на три основных модуля: модуль мониторинга и визуализации параметров, модуль управления исполнительными механизмами, модуль администрирования, логирования и отчётности. Для каждого модуля были определены пользователи, интерфейсные и внутрисистемные функции, построены диаграммы вариантов использования. Также были разработаны диаграммы последовательности для ключевых процессов и диаграмма классов, отражающая основные сущности системы и связи между ними.

В работе был выполнен выбор программно-аппаратной платформы АСУ. Были определены серверное, клиентское и сетевое оборудование (ПЛК, датчики, исполнительные механизмы, SCADA-серверы), операционные системы, СУБД (PostgreSQL + TimescaleDB), SCADA-система (TRACE MODE[7]) и средства

защиты. Также была определена функция, требующая доработки, — автоматическая настройка ПИД-регулятора. Для её реализации был выбран стек разработки (Python + VS Code), представлен исходный код программного модуля и интерфейс его работы.

На основе диаграммы классов была построена ER-диаграмма базы данных. В качестве СУБД была выбрана PostgreSQL. Были определены основные сущности базы данных: климатические зоны, датчики, исполнительные механизмы, ПИД-регуляторы, аварийные события, пользователи, логи, отчёты и настройки системы.

Также была разработана схема сети проектируемой АСУ. В сетевой архитектуре были выделены основные сегменты: полевой уровень (датчики и исполнительные механизмы), контроллерный уровень (ПЛК), серверная зона (SCADA-сервер, сервер БД) и рабочие места пользователей (агроном, инженер, администратор, КИПиА).

Для взаимодействия пользователей с базой данных был разработан программный модуль с Web-интерфейсом. Он обеспечивает просмотр, ввод, редактирование и обработку информации о климатических зонах, растениях и параметрах микроклимата. Разработанный интерфейс позволяет пользователям централизованно получать доступ к данным АСУ через браузер.

На заключительном этапе были рассмотрены вопросы защиты информации. Были выделены основные угрозы безопасности: несанкционированный доступ к SCADA, перехват трафика, подмена команд исполнительным механизмам, вредоносное ПО, отказ оборудования и утечка архивных данных. Для их минимизации предложены средства защиты: разграничение прав доступа, двухфакторная аутентификация, межсетевое экранирование, VPN, антивирусная защита, резервное копирование и журналирование действий пользователей.

Таким образом, в ходе курсовой работы был разработан комплексный проект автоматизированной системы управления поддержанием температуры в оранжерее, включающий описание предметной области, требования, UML-модели, программно-аппаратную платформу, базу данных, сетевую инфраструктуру, Web-интерфейс и средства защиты информации. Проектируемая система позволяет

повысить надёжность поддержания температурного режима, сократить время реакции на отклонения, обеспечить централизованный контроль климатических зон, повысить точность регулирования и снизить энергозатраты на обогрев и вентиляцию.

СПИСОК ЛИТЕРАТУРЫ

1. ГОСТ 34.201-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Виды, комплектность и обозначение документов при создании автоматизированных систем : национальный стандарт Российской Федерации. – URL: https://meganorm.ru/Index/76/76011.htm (дата обращения: 24.02.2026). – Текст: электронный.
2. ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы : национальный стандарт Российской Федерации. – URL: https://meganorm.ru/Index/76/76009.htm (дата обращения: 28.02.2026). – Текст: электронный.
3. ГОСТ 34.601-90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания : межгосударственный стандарт. – URL: https://docs.cntd.ru/document/1200006921 (дата обращения: 02.05.2026). – Текст: электронный.
4. ГОСТ Р МЭК 61131-3-2016. Контроллеры программируемые. Языки программирования : национальный стандарт Российской Федерации. – URL: https://standards.cntd.ru (дата обращения: 10.03.2026). – Текст : электронный.
5. PlantUML Documentation. – URL: https://plantuml.com/ (дата обращения: 26.03.2026). – Текст : электронный.
6. PostgreSQL Documentation. – URL: https://www.postgresql.org/docs/ (дата обращения: 31.03.2026). – Текст: электронный.
7. TRACE MODE SCADA Documentation. – URL: https://www.tracemode.ru/ (дата обращения: 09.04.2026). – Текст: электронный.
8. NIST Special Publication 800-82. Guide to Operational Technology Security. – URL: https://csrc.nist.gov/pubs/sp/800/82/r3/final (дата обращения: 17.04.2026). – Текст: электронный.